

# Отчёт по практической работе №1

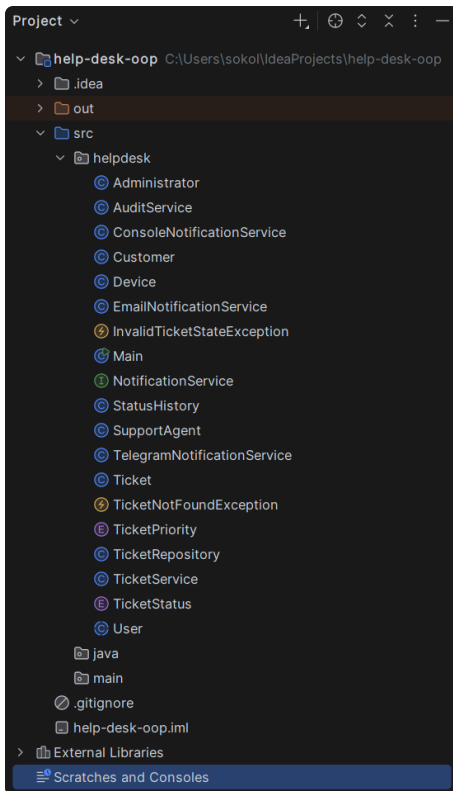
## Объектно-ориентированное программирование на Java. Разработка консольной системы Help Desk

Лёшина Ирина Дмитриевна

2024-ФГИИБ-ИБ-16

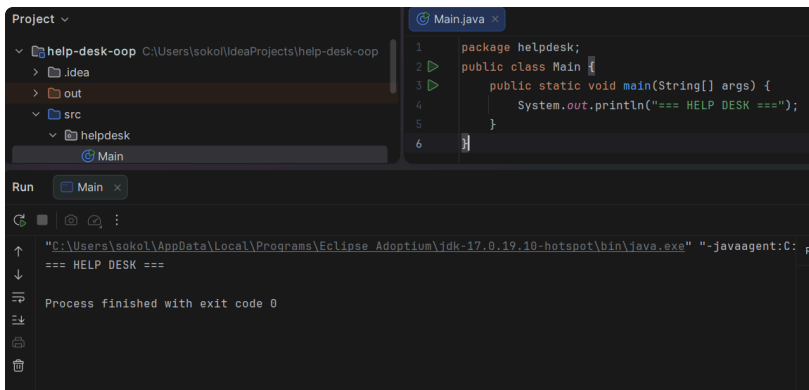
Дата выполнения: 20.09.2026

## 1. Структура проекта:



## 2. Выполнение практической работы

### 1. Создание проекта и точки входа

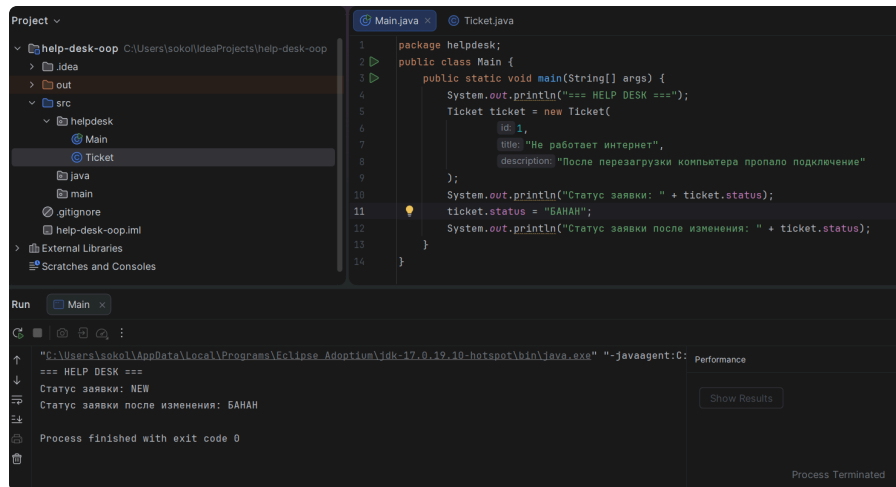


Создан консольный Java-проект на Java 17. Создан класс `Main` с методом `main()`,

который является точкой входа в программу. Для проверки запуска выведено сообщение

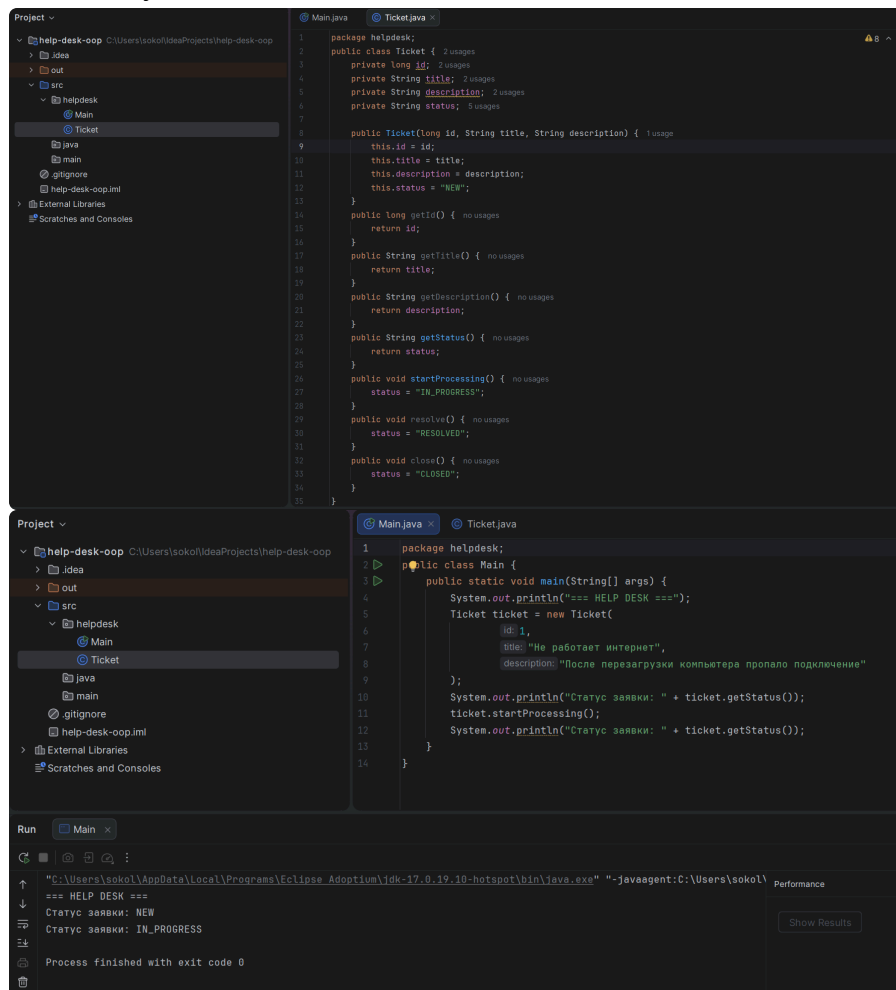
```
=== HELP DESK ===
```

## 2.Первая модель Ticket



Создан класс Ticket, описывающий заявку. Добавлены поля id, title, description и status, конструктор и начальный статус NEW. На этом этапе поля были открытыми, поэтому состояние заявки можно было изменить напрямую. Это показало необходимость использования инкапсуляции.

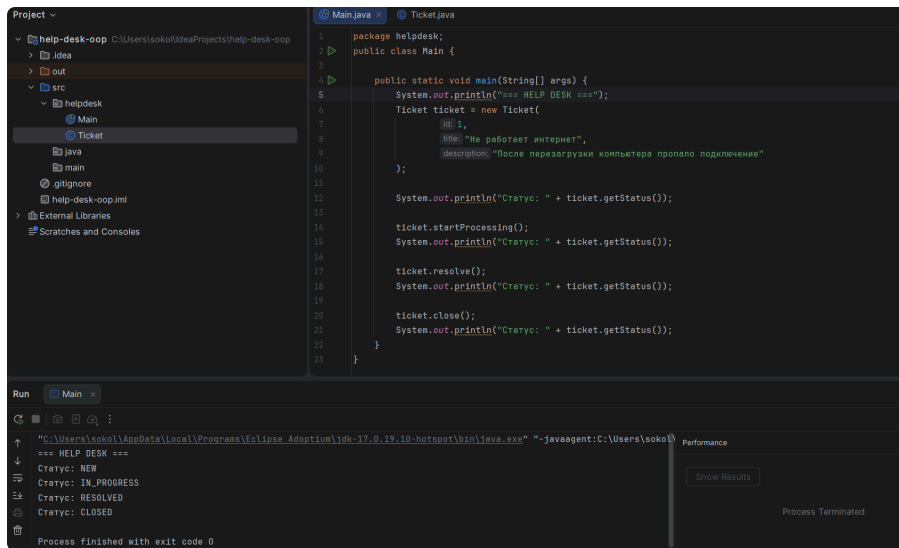
## 3.Инкапсуляция состояния заявки



Поля `Ticket` имеют модификатор `private`, поэтому внешний код не может напрямую изменить состояние заявки. `ticket.close()` лучше, чем `ticket.setStatus("CLOSED")`, потому что метод `close()` передаёт **смысл операции** и позволяет самому объекту осуществлять **контроль состояния**.

#### 4. Контроль допустимых переходов

```
1 package helpdesk;
2 public class Ticket { 2 usages
3     private long id; 2 usages
4     private String title; 2 usages
5     private String description; 2 usages
6     private String status; 8 usages
7
8     public Ticket(long id, String title, String description) { 1 usage
9         this.id = id;
10        this.title = title;
11        this.description = description;
12        this.status = "NEW";
13    }
14
15    public long getId() { no usages
16        return id;
17    }
18
19    public String getTitle() { no usages
20        return title;
21    }
22
23    public String getDescription() { no usages
24        return description;
25    }
26
27    public String getStatus() { 2 usages
28        return status;
29    }
30
31    public void startProcessing() { 1 usage
32        if (!status.equals("NEW")) {
33            System.out.println("Ошибка: в работу можно взять только новую заявку");
34            return;
35        }
36
37        status = "IN_PROGRESS";
38    }
39
40    public void resolve() { no usages
41        if (!status.equals("IN_PROGRESS")) {
42            System.out.println("Ошибка: решить можно только заявку в работе");
43            return;
44        }
45
46        status = "RESOLVED";
47    }
48
49    public void close() { no usages
50        if (!status.equals("RESOLVED")) {
51            System.out.println("Ошибка: закрыть можно только решённую заявку");
52            return;
53        }
54
55        status = "CLOSED";
56    }
57 }
```

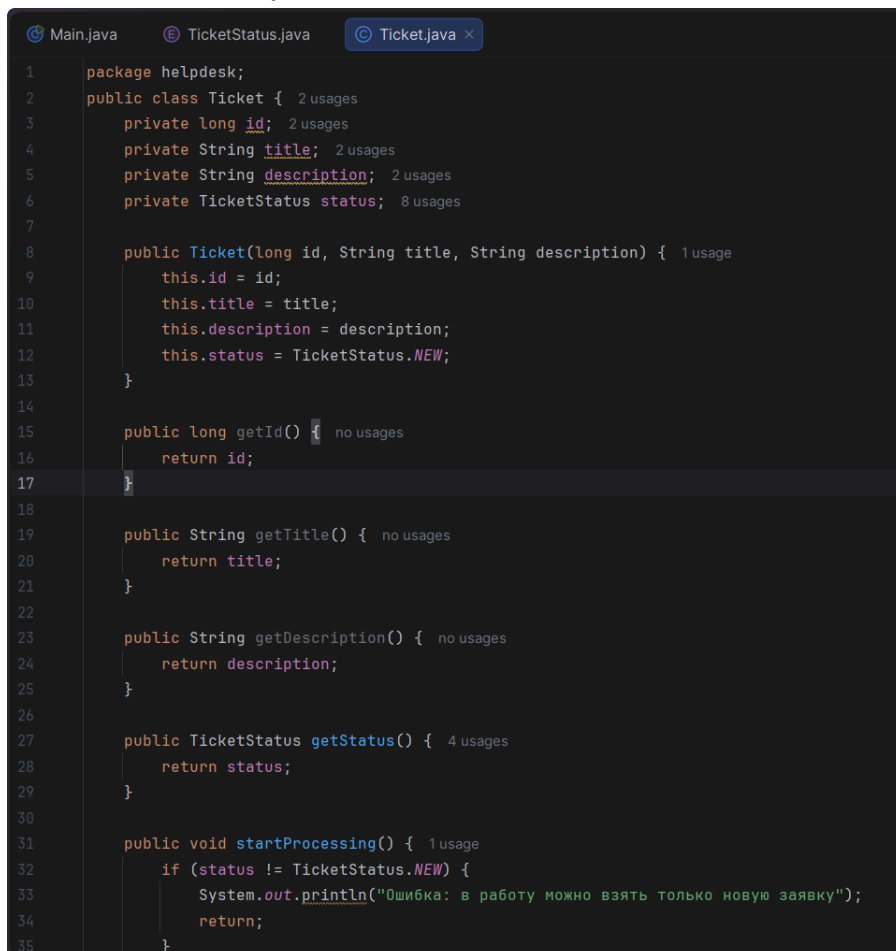


В методы изменения состояния добавлены проверки. `Ticket` самостоятельно контролирует допустимые переходы.

Правильный жизненный цикл:

NEW → IN\_PROGRESS → RESOLVED → CLOSED

## 5.Enum вместо строк



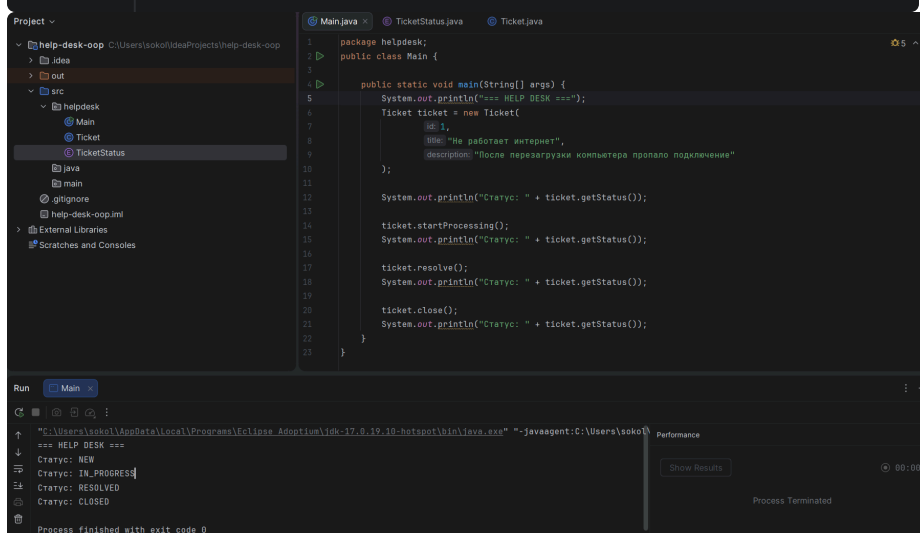
```

36         status = TicketStatus.IN_PROGRESS;
37     }
38
39
40     public void resolve() { 1 usage
41         if (status != TicketStatus.IN_PROGRESS) {
42             System.out.println("Ошибка: решить можно только заявку в работе");
43             return;
44         }
45
46         status = TicketStatus.RESOLVED;
47     }
48
49
50     public void close() { 1 usage
51         if (status != TicketStatus.RESOLVED) {
52             System.out.println("Ошибка: закрыть можно только решённую заявку");
53             return;
54         }
55
56         status = TicketStatus.CLOSED;
57     }
58 }

```

© Main.java    **ⓔ TicketStatus.java** ×    © Ticket.java

```
1 package helpdesk;
2
3 public enum TicketStatus { no usages
4     NEW, no usages
5     IN_PROGRESS, no usages
6     RESOLVED, no usages
7     CLOSED no usages
8 }
9
```



Для статуса заявки создано перечисление `TicketStatus`. Поле `status` в `Ticket` заменено с `String` на `TicketStatus`. Использование `enum` ограничивает возможные значения статуса и исключает ошибки при написании строковых значений.

## 6. Базовый класс User

```
1 package helpdesk;
2 public class User { no usages
3     private long id; 2 usages
4     private String name; 2 usages
5     private String email; 2 usages
6
7     public User(long id, String name, String email) { no usages
8         this.id = id;
9         this.name = name;
10        this.email = email;
11    }
12
13    public long getId() { no usages
14        return id;
15    }
16
17    public String getName() { no usages
18        return name;
19    }
20
21    public String getEmail() { no usages
22        return email;
23    }
24 }
```

Создан базовый класс `User`. В нём определены общие свойства пользователя: `id`, `name` и `email`. Также добавлен конструктор и `getter`-методы.

## 7. Наследование ролей

```
1 package helpdesk;
2 public class Customer extends User { no usages
3     public Customer(long id, String name, String email) { no usages
4         super(id, name, email);
5     }
6 }
7
```

```
1 package helpdesk;
2 public class SupportAgent extends User { no usages
3     public SupportAgent(long id, String name, String email) { no usages
4         super(id, name, email);
5     }
6 }
7
8
9
```

```
Main.java User.java Customer.java SupportAgent.java Administrator.java TicketStatus.java
1 package helpdesk;
2 public class Administrator extends User { no usages
3     public Administrator(long id, String name, String email) { no usages
4         super(id, name, email);
5     }
6 }
7
8
9
```

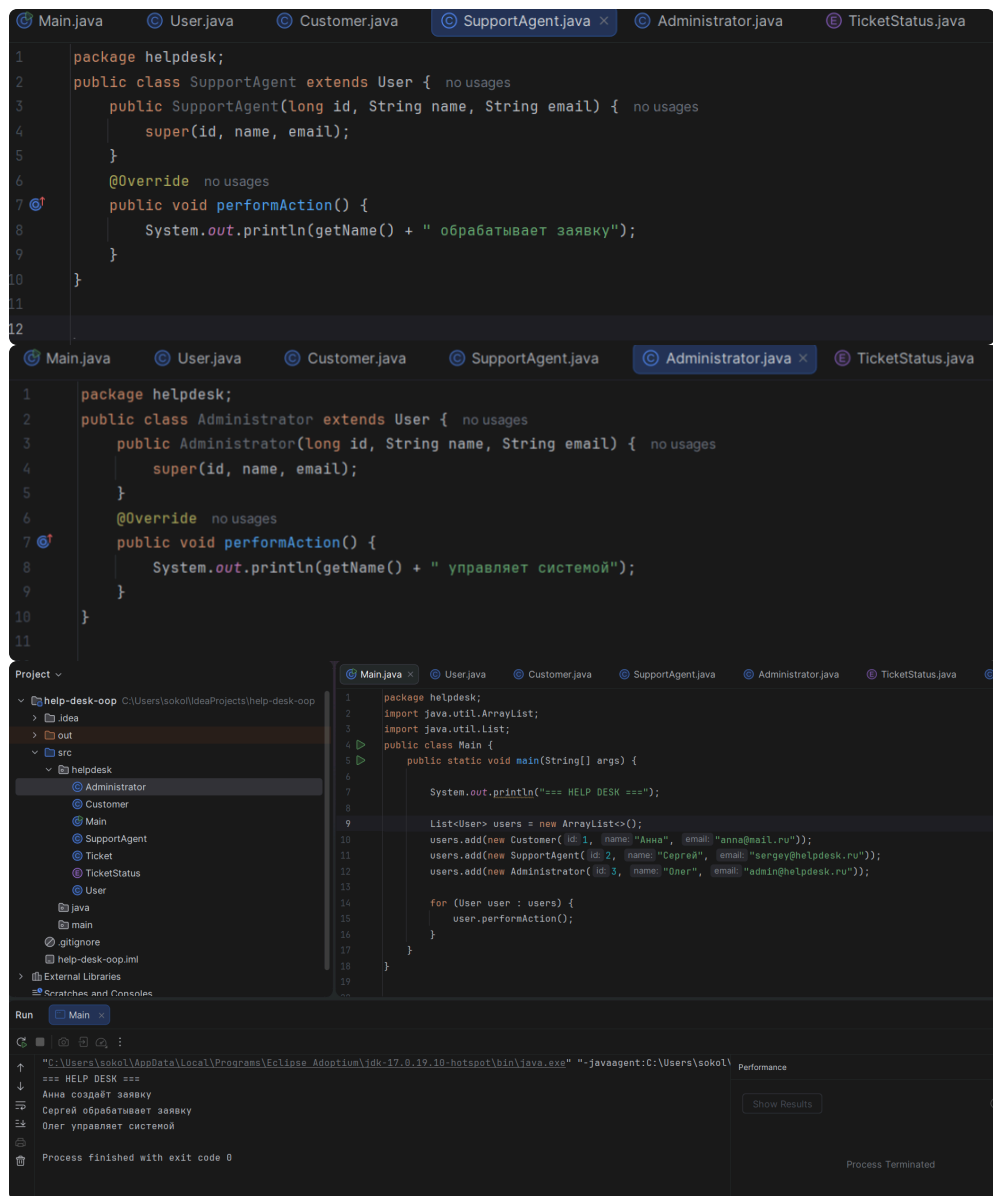
Созданы три класса-наследника. Использовано наследование через `extends`.

- Customer IS-A User
- SupportAgent IS-A User
- Administrator IS-A User

При этом `Ticket` не является пользователем, поэтому наследование `Ticket` от `User` было бы неправильным.

## 8.Переопределение и полиморфизм

```
Main.java User.java Customer.java SupportAgent.java Administrator.java TicketStatus.java
1 package helpdesk;
2 public class User { 3 usages 3 inheritors
3     private long id; 2 usages
4     private String name; 2 usages
5     private String email; 2 usages
6
7     public User(long id, String name, String email) { 3 usages
8         this.id = id;
9         this.name = name;
10        this.email = email;
11    }
12
13    public long getId() { no usages
14        return id;
15    }
16
17    public String getName() { no usages
18        return name;
19    }
20
21    public String getEmail() { no usages
22        return email;
23    }
24
25    public void performAction() { no usages
26        System.out.println("Пользователь работает с системой");
27    }
28 }
29
Main.java User.java Customer.java SupportAgent.java Administrator.java TicketStatus.java
1 package helpdesk;
2 public class Customer extends User { no usages
3     public Customer(long id, String name, String email) { no usages
4         super(id, name, email);
5     }
6     @Override no usages
7     public void performAction() {
8         System.out.println(getName() + " создаёт заявку");
9     }
10 }
11
```



В класс `User` добавлен метод `performAction()`. В классах `Customer`, `SupportAgent` и `Administrator` этот метод переопределён с помощью `@Override`. Затем создан список `List<User>`, содержащий объекты всех трёх ролей. Несмотря на то, что переменная имеет тип `User`, для каждого объекта вызывается своя реализация `performAction()`. Это демонстрирует полиморфизм.

## 9. Абстрактный User

The screenshot shows an IDE with the `User.java` file open. The code defines an abstract class `User` in the `helpdesk` package. It has three private attributes: `id` (long), `name` (String), and `email` (String). The constructor `User(long id, String name, String email)` initializes these attributes. There are three getter methods: `getId()`, `getName()`, and `getEmail()`. The `performAction()` method is declared as abstract. Below the code, the Project view shows the package structure, and the Build Output window displays an error: `java: helpdesk.User is abstract; cannot be instantiated`.

```
1 package helpdesk;
2 public abstract class User { 5 usages 3 inheritors
3     private long id; 2 usages
4     private String name; 2 usages
5     private String email; 2 usages
6
7     public User(long id, String name, String email) { 3 usages
8         this.id = id;
9         this.name = name;
10        this.email = email;
11    }
12
13    public long getId() { no usages
14        return id;
15    }
16
17    public String getName() { 3 usages
18        return name;
19    }
20
21    public String getEmail() { no usages
22        return email;
23    }
24
25    public abstract void performAction(); 1 usage 3 implementations
26 }
27
```

Project view shows the package structure:

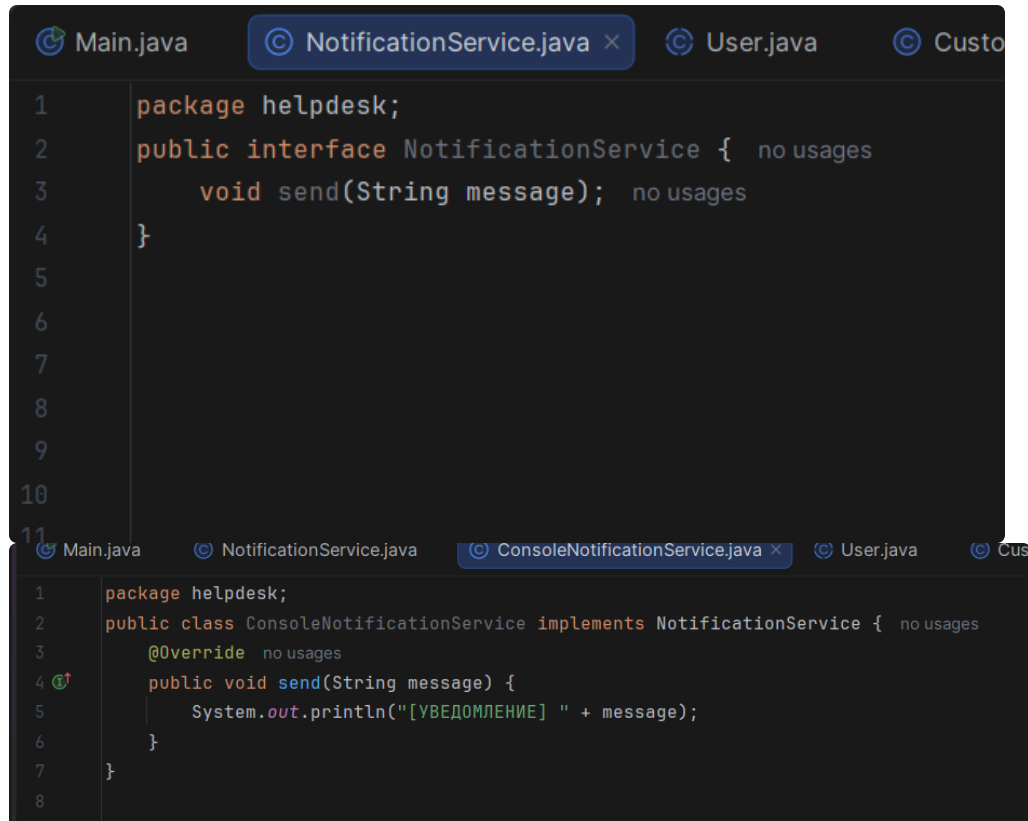
- help-desk-oop
  - idea
  - out
  - src
    - helpdesk
      - Administrator
      - Customer
      - Main
      - SupportAgent
      - Ticket

Build Output window shows the error:

```
java: helpdesk.User is abstract; cannot be instantiated
```

Класс `User` был объявлен абстрактным, метод `performAction()` объявлен абстрактным. После этого была выполнена проверка попыткой создать объект `User` напрямую. Абстрактный класс `User` нельзя использовать для создания объекта напрямую. Конкретные роли пользователей должны реализовать абстрактный метод `performAction()`.

## 10. Интерфейс уведомлений

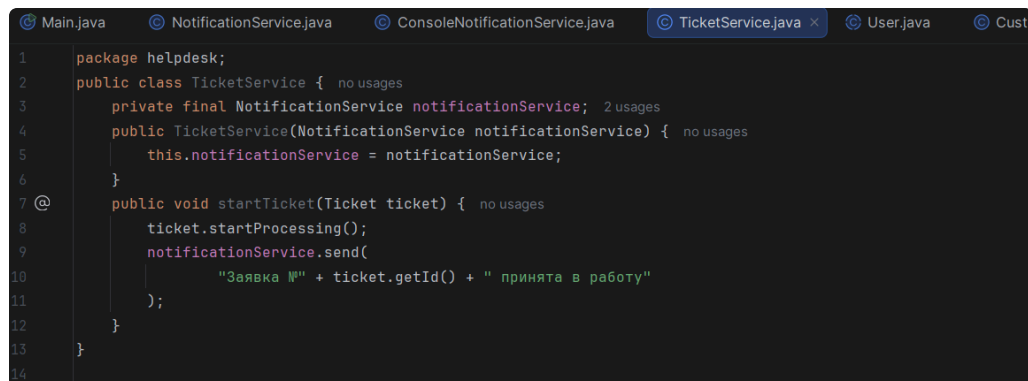


```
1 package helpdesk;
2 public interface NotificationService { no usages
3     void send(String message); no usages
4 }
5
6
7
8
9
10
11
```

```
1 package helpdesk;
2 public class ConsoleNotificationService implements NotificationService { no usages
3     @Override no usages
4     public void send(String message) {
5         System.out.println("[УВЕДОМЛЕНИЕ] " + message);
6     }
7 }
8
```

`NotificationService` является интерфейсом - контрактом для отправки уведомлений. `ConsoleNotificationService` реализует этот контракт и выводит уведомление в консоль.

## 11. TicketService



```
1 package helpdesk;
2 public class TicketService { no usages
3     private final NotificationService notificationService; 2 usages
4     public TicketService(NotificationService notificationService) { no usages
5         this.notificationService = notificationService;
6     }
7     @
8     public void startTicket(Ticket ticket) { no usages
9         ticket.startProcessing();
10        notificationService.send(
11            "Заявка №" + ticket.getId() + " принята в работу"
12        );
13    }
14 }
```

```
1 package helpdesk;
2 import java.util.ArrayList;
3 import java.util.List;
4 public class Main {
5     public static void main(String[] args) {
6         System.out.println("=== HELP DESK ===");
7
8         List<User> users = new ArrayList<>();
9         users.add(new Customer( id: 1, name: "Анна", email: "anna@mail.ru"));
10        users.add(new SupportAgent( id: 2, name: "Сергей", email: "sergey@helpdesk.ru"));
11        users.add(new Administrator( id: 3, name: "Олег", email: "admin@helpdesk.ru"));
12
13        for (User user : users) {
14            user.performAction();
15        }
16
17        Ticket ticket = new Ticket(
18            id: 1,
19            title: "Не работает интернет",
20            description: "После перезагрузки компьютера пропало подключение"
21        );
22
23        NotificationService notificationService =
24            new ConsoleNotificationService();
25
26        TicketService ticketService =
27            new TicketService(notificationService);
28
29        ticketService.startTicket(ticket);
30
31        System.out.println("Статус: " + ticket.getStatus());
32    }
33 }
34
```

Run Main x

↑  
↓  
⇌  
⇕  
🖨  
🗑

```
"C:\Users\sokol\AppData\Local\Programs\Eclipse Adoptium\
=== HELP DESK ===
Анна создаёт заявку
Сергей обрабатывает заявку
Олег управляет системой
[УВЕДОМЛЕНИЕ] Заявка №1 принята в работу
Статус: IN_PROGRESS

Process finished with exit code 0
```

Создан сервис `TicketService`, который выполняет действия с заявкой и отправляет уведомления. Сервис получает `NotificationService` через конструктор: `public TicketService(NotificationService notificationService)`. При принятии заявки в работу меняется её статус и отправляется уведомление. `TicketService` зависит от интерфейса `NotificationService`, а не от конкретного класса.

## 12.Репозиторий заявок

```
Main.java TicketRepository.java NotificationService.java ConsoleNotificationService.java TicketServ

1 package helpdesk;
2 import java.util.ArrayList;
3 import java.util.List;
4 public class TicketRepository { no usages
5     private final List<Ticket> tickets = new ArrayList<>(); 2 usages
6
7     public void add(Ticket ticket) { no usages
8         tickets.add(ticket);
9     }
10    public List<Ticket> findAll() { no usages
11        return new ArrayList<>(tickets);
12    }
13 }
14

Main.java x TicketRepository.java NotificationService.java ConsoleNotificationService.java TicketServ

1 package helpdesk;
2 import java.util.ArrayList;
3 import java.util.List;
4 public class Main {
5     public static void main(String[] args) {
6
7         System.out.println("=== HELP DESK ===");
8
9         List<User> users = new ArrayList<>();
10        users.add(new Customer( id: 1, name: "Анна", email: "anna@mail.ru"));
11        users.add(new SupportAgent( id: 2, name: "Сергей", email: "sergey@helpdesk.ru"));
12        users.add(new Administrator( id: 3, name: "Олег", email: "admin@helpdesk.ru"));
13
14        for (User user : users) {
15            user.performAction();
16        }
17
18        Ticket ticket = new Ticket(
19            id: 1,
20            title: "Не работает интернет",
21            description: "После перезагрузки компьютера пропало подключение"
22        );
23
24        NotificationService notificationService =
25            new ConsoleNotificationService();
26
27        TicketService ticketService =
28            new TicketService(notificationService);
29
30        ticketService.startTicket(ticket);
31
32        System.out.println("Статус: " + ticket.getStatus());
33
34        TicketRepository repository = new TicketRepository();
35    }
```

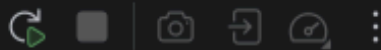
```

36         Ticket ticket2 = new Ticket(
37             id: 2,
38             title: "Ошибка приложения",
39             description: "Приложение не запускается"
40         );
41
42         Ticket ticket3 = new Ticket(
43             id: 3,
44             title: "Не печатает принтер",
45             description: "Принтер не печатает документы"
46         );
47
48         repository.add(ticket);
49         repository.add(ticket2);
50         repository.add(ticket3);
51
52         System.out.println("Все заявки:");
53
54         for (Ticket t : repository.findAll()) {
55             System.out.println(
56                 "#" + t.getId()
57                 + " " + t.getTitle()
58                 + " | " + t.getStatus()
59             );
60         }
61     }
62 }

```

Run

Main ×



```

↑    === HELP DESK ===
↓    Анна создаёт заявку
⇌   Сергей обрабатывает заявку
⇌   Олег управляет системой
⇌   [УВЕДОМЛЕНИЕ] Заявка №1 принята в работу
⇌   Статус: IN_PROGRESS
🖨   Все заявки:
🗑   #1 Не работает интернет | IN_PROGRESS
    #2 Ошибка приложения | NEW
    #3 Не печатает принтер | NEW

    Process finished with exit code 0

```

Создан класс `TicketRepository`, который хранит заявки в коллекции `List<Ticket>`.

Добавлены методы: add(Ticket ticket) для добавления заявки и findAll() для получения списка заявок. Для проверки были добавлены три заявки.

### 3. Итоговый обязательный сценарий

```

Main.java TicketRepository.java NotificationService.java ConsoleNotificationService.java TicketService.java
1 package helpdesk;
2 public class TicketService { 2 usages
3     private final NotificationService notificationService; 4 usages
4     public TicketService(NotificationService notificationService) { 1 usage
5         this.notificationService = notificationService;
6     }
7     @
8     public void startTicket(Ticket ticket) { 1 usage
9         ticket.startProcessing();
10
11         notificationService.send(
12             "Заявка №" + ticket.getId() + " принята в работу"
13         );
14     }
15     @
16     public void resolveTicket(Ticket ticket) { no usages
17         ticket.resolve();
18         notificationService.send(
19             "По заявке №" + ticket.getId() + " найдено решение"
20         );
21     }
22     @
23     public void closeTicket(Ticket ticket) { no usages
24         ticket.close();
25         notificationService.send(
26             "Заявка №" + ticket.getId() + " закрыта"
27         );
28     }
29 }
30
Main.java x TicketRepository.java NotificationService.java ConsoleNotificationService.java TicketSer
1 package helpdesk;
2 import java.util.ArrayList;
3 import java.util.List;
4 public class Main {
5     public static void main(String[] args) {
6         System.out.println("=== HELP DESK ===");
7
8         Customer customer = new Customer(
9             id: 1,
10             name: "Анна Петрова",
11             email: "anna@mail.ru"
12         );
13
14         SupportAgent supportAgent = new SupportAgent(
15             id: 2,
16             name: "Сергей Иванов",
17             email: "sergey@helpdesk.ru"
18         );
19
20         System.out.println("Клиент: " + customer.getName());
21
22         Ticket ticket = new Ticket(
23             id: 1,
24             title: "Не работает Wi-Fi",
25             description: "После подключения к сети интернет не работает"
26         );
27
28         System.out.println(
29             "Заявка #" + ticket.getId()
30             + ": " + ticket.getTitle()
31             + " | " + ticket.getStatus()
32         );
33     }
34 }
```

```

34     NotificationService notificationService =
35         new ConsoleNotificationService();
36     TicketService ticketService =
37         new TicketService(notificationService);
38     ticketService.startTicket(ticket);
39     System.out.println("Статус: " + ticket.getStatus());
40     ticketService.resolveTicket(ticket);
41     System.out.println("Статус: " + ticket.getStatus());
42     ticketService.closeTicket(ticket);
43     System.out.println("Статус: " + ticket.getStatus());
44     TicketRepository repository = new TicketRepository();
45     repository.add(ticket);
46     System.out.println("Полный список заявок:");
47
48     for (Ticket t : repository.findAll()) {
49         System.out.println(
50             "#" + t.getId()
51             + " " + t.getTitle()
52             + " | " + t.getStatus()
53         );
54     }
55 }
56 }

```

Run Main x

```

"C:\Users\sokol\AppData\Local\Programs\Eclipse
=== HELP DESK ===
Клиент: Анна Петрова
Заявка #1: Не работает Wi-Fi | NEW
[УВЕДОМЛЕНИЕ] Заявка №1 принята в работу
Статус: IN_PROGRESS
[УВЕДОМЛЕНИЕ] По заявке №1 найдено решение
Статус: RESOLVED
[УВЕДОМЛЕНИЕ] Заявка №1 закрыта
Статус: CLOSED
Полный список заявок:
#1 Не работает Wi-Fi | CLOSED

Process finished with exit code 0

```

Были созданы клиент **Анна Петрова**, специалист поддержки **Сергей Иванов** и заявка «**Не работает Wi-Fi**». При создании заявки её начальный статус установлен в `NEW`. Затем заявка последовательно прошла все этапы жизненного цикла:

`NEW` → `IN_PROGRESS` → `RESOLVED` → `CLOSED`

При каждом успешном изменении статуса `TicketService` отправлял уведомление через `NotificationService`. После закрытия заявка была добавлена в `TicketRepository`, после чего выведен полный список сохранённых заявок.

## Вывод

Итоговый сценарий выполнен успешно. Заявка создаётся со статусом `NEW`, проходит допустимые этапы обработки и закрывается со статусом `CLOSED`. Уведомления выводятся при каждом успешном переходе, а заявка сохраняется в репозитории.

## 4. Четыре принципа ООП в проекте

| Принцип      | Что найти в проекте                                                                                             | Ваш пример                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Инкапсуляция | <code>private</code> -поля <code>Ticket</code> ; изменение статуса только через методы                          | <pre>private TicketStatus status; public TicketStatus getStatus() { return status; } public void close() { if (status != TicketStatus.RESOLVED) { System.out.println( "Ошибка: закрыть можно только решённую заявку" ); return; } status = TicketStatus.CLOSED; }</pre> В классе <code>Ticket</code> поля объекта закрыты с помощью модификатора <code>private</code> . Изменение статуса выполняется через методы, которые контролируют состояние заявки.                                                                                                                    |
| Наследование | <code>Customer</code> , <code>SupportAgent</code> , <code>Administrator</code> наследуются от <code>User</code> | <pre>public class Customer extends User { public Customer(long id, String name, String email) { super(id, name, email); } }</pre> В проекте классы <code>Customer</code> , <code>SupportAgent</code> и <code>Administrator</code> наследуются от базового класса <code>User</code> . Наследование позволяет классам-наследникам использовать общие свойства и методы <code>User</code> . Между классами существует отношение IS-A.                                                                                                                                            |
| Полиморфизм  | <code>List&lt;User&gt;</code> и вызов <code>user.performAction()</code>                                         | <pre>List&lt;User&gt; users = new ArrayList&lt;&gt;(); users.add(new Customer(1, "Анна", "anna@mail.ru")); users.add(new SupportAgent(2, "Сергей", "sergey@helpdesk.ru")); users.add(new Administrator(3, "Олег", "admin@helpdesk.ru")); for (User user : users) { user.performAction(); }</pre> Для хранения пользователей используется список типа <code>List&lt;User&gt;</code> . В список добавляются объекты разных классов, а действие вызывается одним и тем же методом. Один вызов <code>user.performAction()</code> приводит к выполнению разных реализаций метода в |

| Принцип           | Что найти в проекте                                      | Ваш пример                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------------|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                   |                                                          | зависимости от типа объекта. Это демонстрирует полиморфизм.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Абстракция</b> | <pre>abstract User ; интерфейс NotificationService</pre> | <p>Базовый класс <code>User</code> объявлен абстрактным, а метод <code>performAction()</code> задаёт обязательное действие для классов-наследников. <code>public abstract class User { public abstract void performAction(); }</code> Также в проекте используется интерфейс <code>NotificationService</code>: <code>public interface NotificationService { void send(String message); }</code> Абстракция выделяет общие характеристики и поведение, не раскрывая конкретную реализацию. <code>User</code> задаёт общую основу для ролей пользователей, а <code>NotificationService</code> определяет контракт отправки уведомлений.</p> |

## 5.Самостоятельная работа

№1

```

1 package helpdesk;
2 public enum TicketPriority { no usages
3     LOW, no usages
4     MEDIUM, no usages
5     HIGH, no usages
6     CRITICAL no usages
7 }
8
9
10
11
12
13
14
15
16
17 package helpdesk;
18 public class Ticket { 9 usages
19     private long id; 2 usages
20     private String title; 2 usages
21     private String description; 2 usages
22     private TicketStatus status; 8 usages
23     private TicketPriority priority; 2 usages
24
25     public Ticket(long id, String title, String description, TicketPriority priority) { 1 usage
26         this.id = id;
27         this.title = title;
28         this.description = description;
29         this.status = TicketStatus.NEW;
30         this.priority = priority;
31     }
32
33     public long getId() { 5 usages
34         return id;
35     }
36
37     public String getTitle() { 2 usages
38         return title;
39     }
40
41     public String getDescription() { no usages
42         return description;
43     }
44
45     public TicketStatus getStatus() { 5 usages
46         return status;
47     }
48 }

```

```

33     public TicketPriority getPriority() { no usages
34         return priority;
35     }
36
37     public void startProcessing() { 1 usage
38         if (status != TicketStatus.NEW) {
39             System.out.println("Ошибка: в работу можно взять только новую заявку");
40             return;
41         }
42
43         status = TicketStatus.IN_PROGRESS;
44     }
45
46     public void resolve() { 1 usage
47         if (status != TicketStatus.IN_PROGRESS) {
48             System.out.println("Ошибка: решить можно только заявку в работе");
49             return;
50         }
51
52         status = TicketStatus.RESOLVED;
53     }
54
55     public void close() { 1 usage
56         if (status != TicketStatus.RESOLVED) {
57             System.out.println("Ошибка: закрыть можно только решённую заявку");
58             return;
59         }
60
61         status = TicketStatus.CLOSED;
62     }

```

```

Main.java x TicketRepository.java NotificationService.java ConsoleNotificationS
1 package helpdesk;
2 public class Main {
3     public static void main(String[] args) {
4
5         System.out.println("=== HELP DESK ===");
6
7         Customer customer = new Customer(
8             id: 1,
9             name: "Анна Петрова",
10            email: "anna@mail.ru"
11        );
12
13        SupportAgent supportAgent = new SupportAgent(
14            id: 2,
15            name: "Сергей Иванов",
16            email: "sergey@helpdesk.ru"
17        );
18
19        System.out.println("Клиент: " + customer.getName());
20
21        Ticket ticket = new Ticket(
22            id: 1,
23            title: "Не работает Wi-Fi",
24            description: "После подключения к сети интернет не работает",
25            TicketPriority.HIGH
26        );
27
28        System.out.println(
29            "Заявка #" + ticket.getId()
30            + ": " + ticket.getTitle()
31            + " | " + ticket.getStatus()
32            + " | Приоритет: " + ticket.getPriority()
33        );

```

```

63     );
64
65     NotificationService notificationService =
66         new ConsoleNotificationService();
67
68     TicketService ticketService =
69         new TicketService(notificationService);
70
71     ticketService.startTicket(ticket);
72     System.out.println("Статус: " + ticket.getStatus());
73
74     ticketService.resolveTicket(ticket);
75     System.out.println("Статус: " + ticket.getStatus());
76
77     ticketService.closeTicket(ticket);
78     System.out.println("Статус: " + ticket.getStatus());
79
80     TicketRepository repository = new TicketRepository();
81     repository.add(ticket);
82
83     System.out.println("Полный список заявок:");
84
85     for (Ticket t : repository.findAll()) {
86         System.out.println(
87             "#" + t.getId()
88             + " " + t.getTitle()
89             + " | " + t.getStatus()
90             + " | Приоритет: " + t.getPriority()
91         );
92     }
93 }
94 }

```

Run Main x

"C:\Users\sokol\AppData\Local\Programs\Eclipse Adoptio  
 === HELP DESK ===  
 Клиент: Анна Петрова  
 Заявка #1: Не работает Wi-Fi | NEW | Приоритет: HIGH  
 [УВЕДОМЛЕНИЕ] Заявка №1 принята в работу  
 Статус: IN\_PROGRESS  
 [УВЕДОМЛЕНИЕ] По заявке №1 найдено решение  
 Статус: RESOLVED  
 [УВЕДОМЛЕНИЕ] Заявка №1 закрыта  
 Статус: CLOSED  
 Полный список заявок:  
 #1 Не работает Wi-Fi | CLOSED | Приоритет: HIGH  
  
 Process finished with exit code 0

В проект добавлено перечисление `TicketPriority` с четырьмя уровнями приоритета. В класс `Ticket` добавлено поле приоритета. Приоритет передаётся при создании заявки и доступен только для чтения.

№2

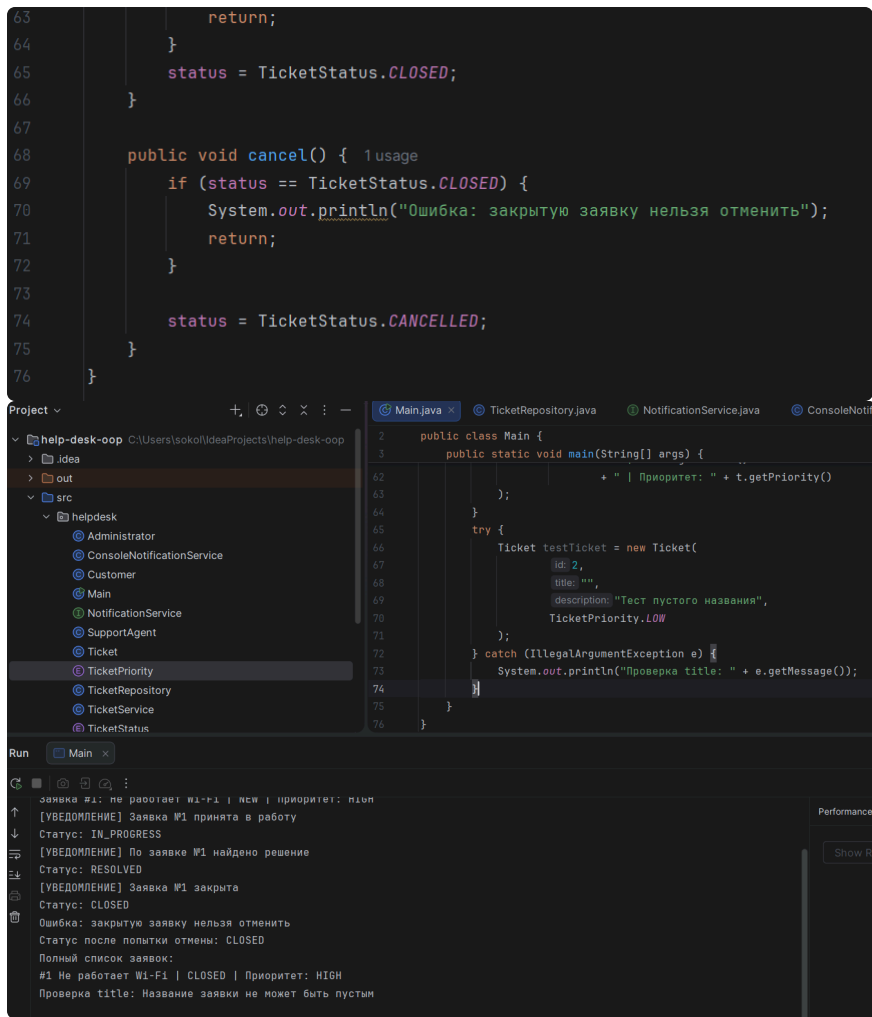
```
Customer.java SupportAgent.java TicketPriority.java Administrator.java TicketStatus.java x
1 package helpdesk;
2 public enum TicketStatus { 9 usages
3     NEW, 2 usages
4     IN_PROGRESS, 2 usages
5     RESOLVED, 2 usages
6     CLOSED, 1 usage
7     CANCELLED no usages
8 }
9
10
11
Customer.java SupportAgent.java TicketPriority.java Administrator.java TicketStatus.java Ticket.java x
2 public class Ticket { 9 usages
3
45
46 public void resolve() { 1 usage
47     if (status != TicketStatus.IN_PROGRESS) {
48         System.out.println("Ошибка: решить можно только заявку в работе");
49         return;
50     }
51
52     status = TicketStatus.RESOLVED;
53 }
54
55 public void close() { 1 usage
56     if (status != TicketStatus.RESOLVED) {
57         System.out.println("Ошибка: закрыть можно только решённую заявку");
58         return;
59     }
60
61     status = TicketStatus.CLOSED;
62 }
63
64 public void cancel() { no usages
65     if (status == TicketStatus.CLOSED) {
66         System.out.println("Ошибка: закрытую заявку нельзя отменить");
67         return;
68     }
69
70     status = TicketStatus.CANCELLED;
71 }
72 }
73
Main.java x TicketRepository.java NotificationService.java ConsoleNotificationService.java
2 public class Main {
3     public static void main(String[] args) {
44
45         ticketService.resolveTicket(ticket);
46         System.out.println("Статус: " + ticket.getStatus());
47
48         ticketService.closeTicket(ticket);
49         System.out.println("Статус: " + ticket.getStatus());
50         ticket.cancel();
51         System.out.println("Статус после попытки отмены: " + ticket.getStatus());
52
53         TicketRepository repository = new TicketRepository();
54         repository.add(ticket);
55
56         System.out.println("Полный список заявок:");
57
58         for (Ticket t : repository.findAll()) {
59
60             ticketService.closeTicket(ticket);
61             System.out.println("Статус: " + ticket.getStatus());
62             ticket.cancel();
63             System.out.println("Статус после попытки отмены: " + ticket.getStatus());
64         }
65     }
66 }
67
helpdesk
  Administrator
  ConsoleNotificationService
  Customer
  Main
  NotificationService
  TicketRepository
  TicketStatus
  Ticket

Run Main x
C:\Users\sokol\AppData\Local\Programs\Eclipse_Adoptium\jdk-17.0.19-hotspot\bin\java.exe -javaagent:C:\Users\sokol\
Performance
HELP DESK
Клиент: Анна Петрова
Заявка #1: Не работает Wi-Fi | NEW | Приоритет: HIGH
[УВЕДОМЛЕНИЕ] Заявка #1 принята в работу
Статус: IN_PROGRESS
[УВЕДОМЛЕНИЕ] По заявке #1 найдено решение
Статус: RESOLVED
[УВЕДОМЛЕНИЕ] Заявка #1 закрыта
Статус: CLOSED
Ошибка: закрытую заявку нельзя отменить
Статус после попытки отмены: CLOSED
Полный список заявок:
#1 Не работает Wi-Fi | CLOSED | Приоритет: HIGH
Process finished with exit code 0
```

В TicketStatus добавлен новый статус CANCELLED. В Ticket добавлен метод cancel(), который запрещает отмену уже закрытой заявки.

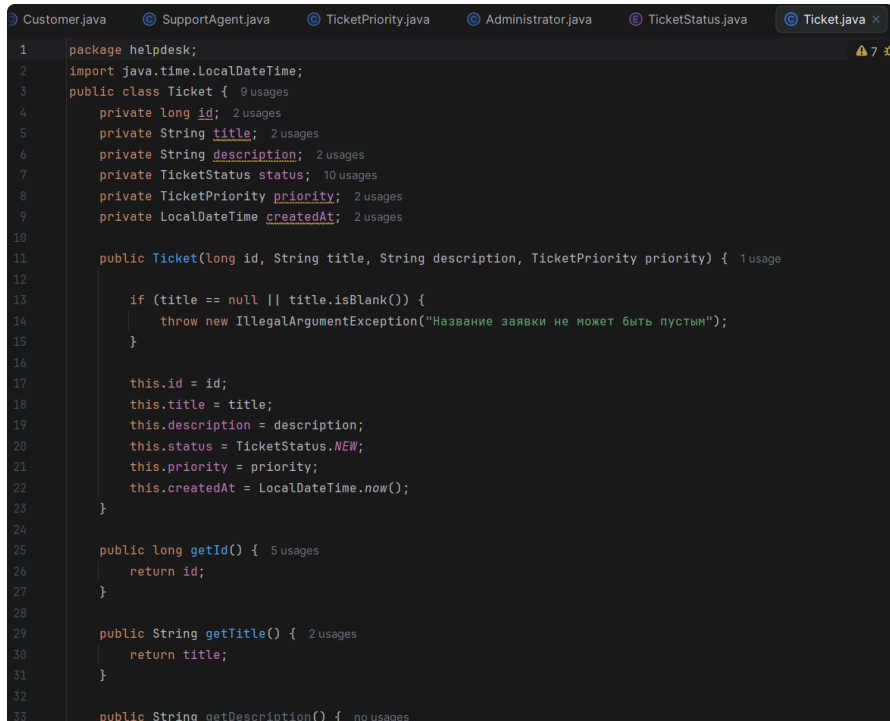
### №3

```
Customer.java SupportAgent.java TicketPriority.java Administrator.java TicketStatus.java Ticket.java
1 package helpdesk;
2 public class Ticket { 0 usages
3     private long id; 2 usages
4     private String title; 2 usages
5     private String description; 2 usages
6     private TicketStatus status; 10 usages
7     private TicketPriority priority; 2 usages
8
9     public Ticket(long id, String title, String description, TicketPriority priority) { 1 usage
10
11         if (title == null || title.isBlank()) {
12             throw new IllegalArgumentException("Название заявки не может быть пустым");
13         }
14
15         this.id = id;
16         this.title = title;
17         this.description = description;
18         this.status = TicketStatus.NEW;
19         this.priority = priority;
20     }
21
22     public long getId() { 5 usages
23         return id;
24     }
25
26     public String getTitle() { 2 usages
27         return title;
28     }
29
30     public String getDescription() { no usages
31         return description;
32     }
33
34     public TicketStatus getStatus() { 6 usages
35         return status;
36     }
37
38     public TicketPriority getPriority() { 2 usages
39         return priority;
40     }
41
42     public void startProcessing() { 1 usage
43         if (status != TicketStatus.NEW) {
44             System.out.println("Ошибка: в работу можно взять только новую заявку");
45             return;
46         }
47
48         status = TicketStatus.IN_PROGRESS;
49     }
50
51     public void resolve() { 1 usage
52         if (status != TicketStatus.IN_PROGRESS) {
53             System.out.println("Ошибка: решить можно только заявку в работе");
54             return;
55         }
56
57         status = TicketStatus.RESOLVED;
58     }
59
60     public void close() { 1 usage
61         if (status != TicketStatus.RESOLVED) {
62             System.out.println("Ошибка: закрыть можно только решённую заявку");
63             return;
64         }
65     }
66 }
```



Добавлена валидация названия заявки. Пустое название считается недопустимым и вызывает `IllegalArgumentException`.

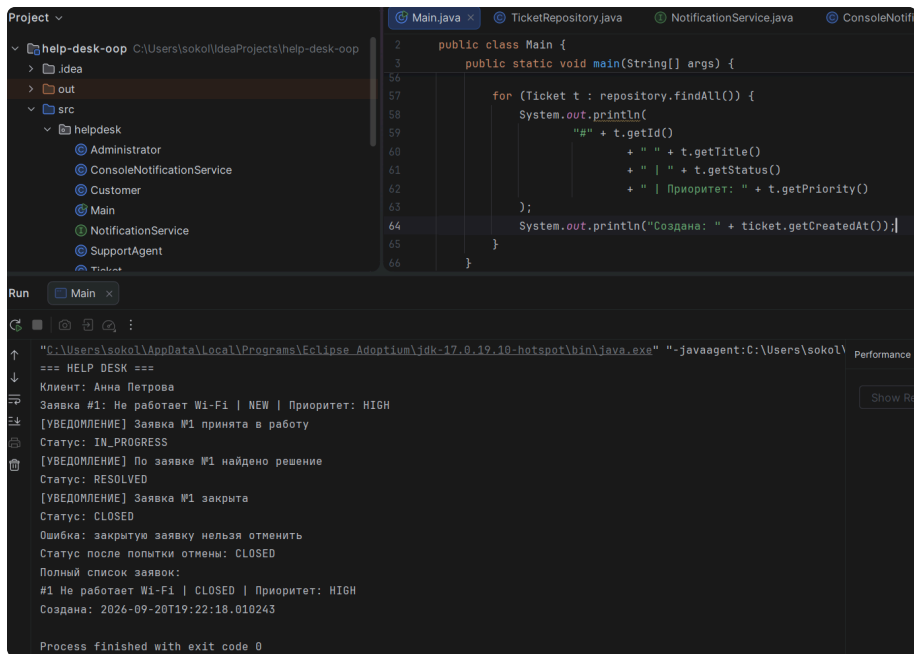
## №4



```

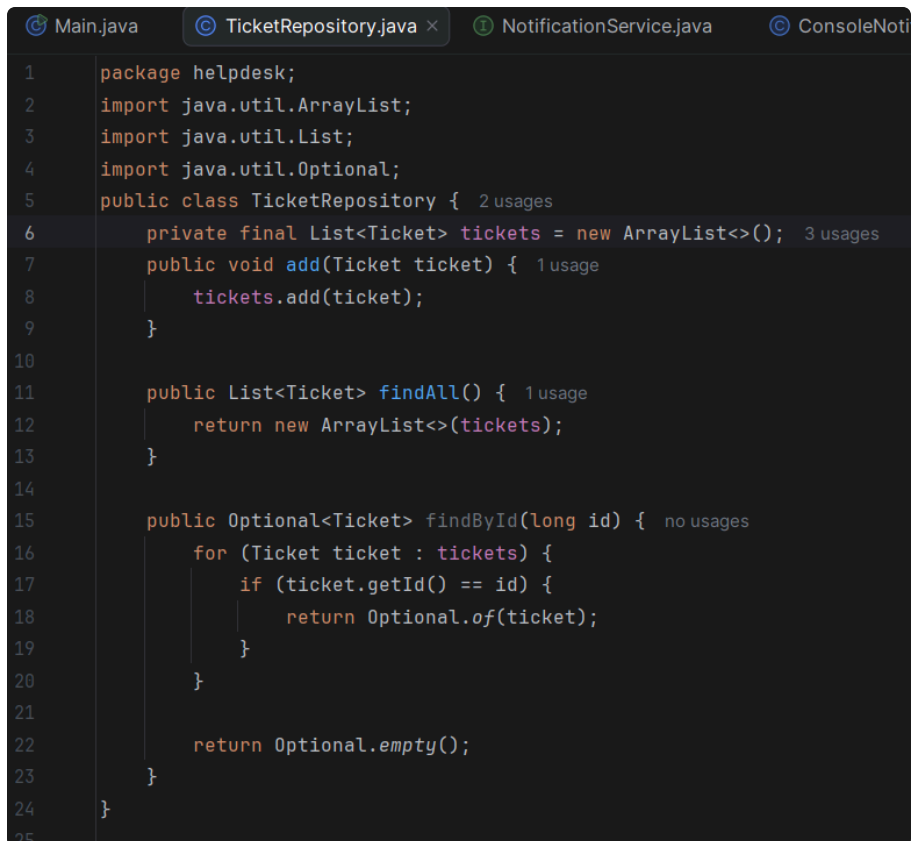
33     public String getDescription() { no usages
34         return description;
35     }
36
37     public TicketStatus getStatus() { 6 usages
38         return status;
39     }
40
41     public TicketPriority getPriority() { 2 usages
42         return priority;
43     }
44
45     public LocalDateTime getCreatedAt() { no usages
46         return createdAt;
47     }
48
49     public void startProcessing() { 1 usage
50         if (status != TicketStatus.NEW) {
51             System.out.println("Ошибка: в работу можно взять только новую заявку");
52             return;
53         }
54
55         status = TicketStatus.IN_PROGRESS;
56     }
57
58     public void resolve() { 1 usage
59         if (status != TicketStatus.IN_PROGRESS) {
60             System.out.println("Ошибка: решить можно только заявку в работе");
61             return;
62         }
63         System.out.println("Ошибка: решить можно только заявку в работе");
64         return;
65     }
66
67     public void close() { 1 usage
68         if (status != TicketStatus.RESOLVED) {
69             System.out.println("Ошибка: закрыть можно только решённую заявку");
70             return;
71         }
72
73         status = TicketStatus.CLOSED;
74     }
75
76     public void cancel() { 1 usage
77         if (status == TicketStatus.CLOSED) {
78             System.out.println("Ошибка: закрытую заявку нельзя отменить");
79             return;
80         }
81
82         status = TicketStatus.CANCELLED;
83     }
84 }

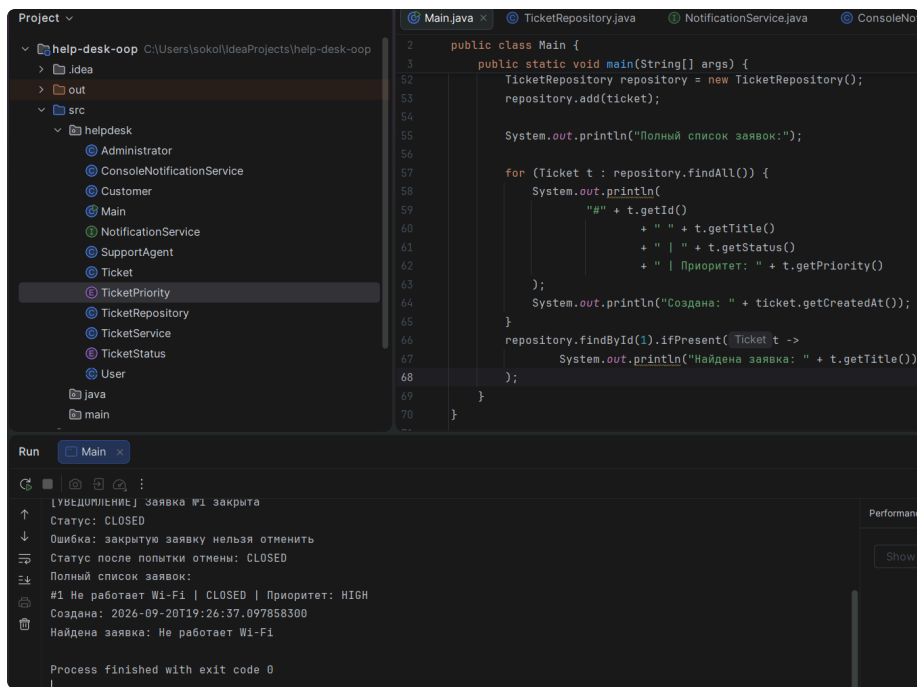
```



В класс `Ticket` добавлено поле `createdAt` типа `LocalDateTime`. Время автоматически фиксируется при создании заявки.

## №7





В `TicketRepository` добавлен метод `findById()` . Если заявка найдена, возвращается `Optional<Ticket>` , а не `null` .

The image shows an IDE with the following components:

- Editor:** Displays `TicketRepository.java` with the following code:
 

```

1 package helpdesk;
2 import java.util.ArrayList;
3 import java.util.List;
4 import java.util.Optional;
5 public class TicketRepository {
6     private final List<Ticket> tickets = new ArrayList<>();
7     public void add(Ticket ticket) {
8         tickets.add(ticket);
9     }
10
11     public List<Ticket> findAll() {
12         return new ArrayList<>(tickets);
13     }
14
15     public Optional<Ticket> findById(long id) {
16         for (Ticket ticket : tickets) {
17             if (ticket.getId() == id) {
18                 return Optional.of(ticket);
19             }
20         }
21         return Optional.empty();
22     }
23
24     public List<Ticket> findByStatus(TicketStatus status) {
25         List<Ticket> result = new ArrayList<>();
26
27         for (Ticket ticket : tickets) {
28             if (ticket.getStatus() == status) {
29                 result.add(ticket);
30             }
31         }
32
33         return result;
34     }
35 }

```
- Project Explorer:** Shows the project structure:
  - help-desk-oop
    - src
      - helpdesk
        - Administrator
        - ConsoleNotificationService
        - Customer
        - Main
        - NotificationService
- Run Console:** Displays the output of the `Main` class:
 

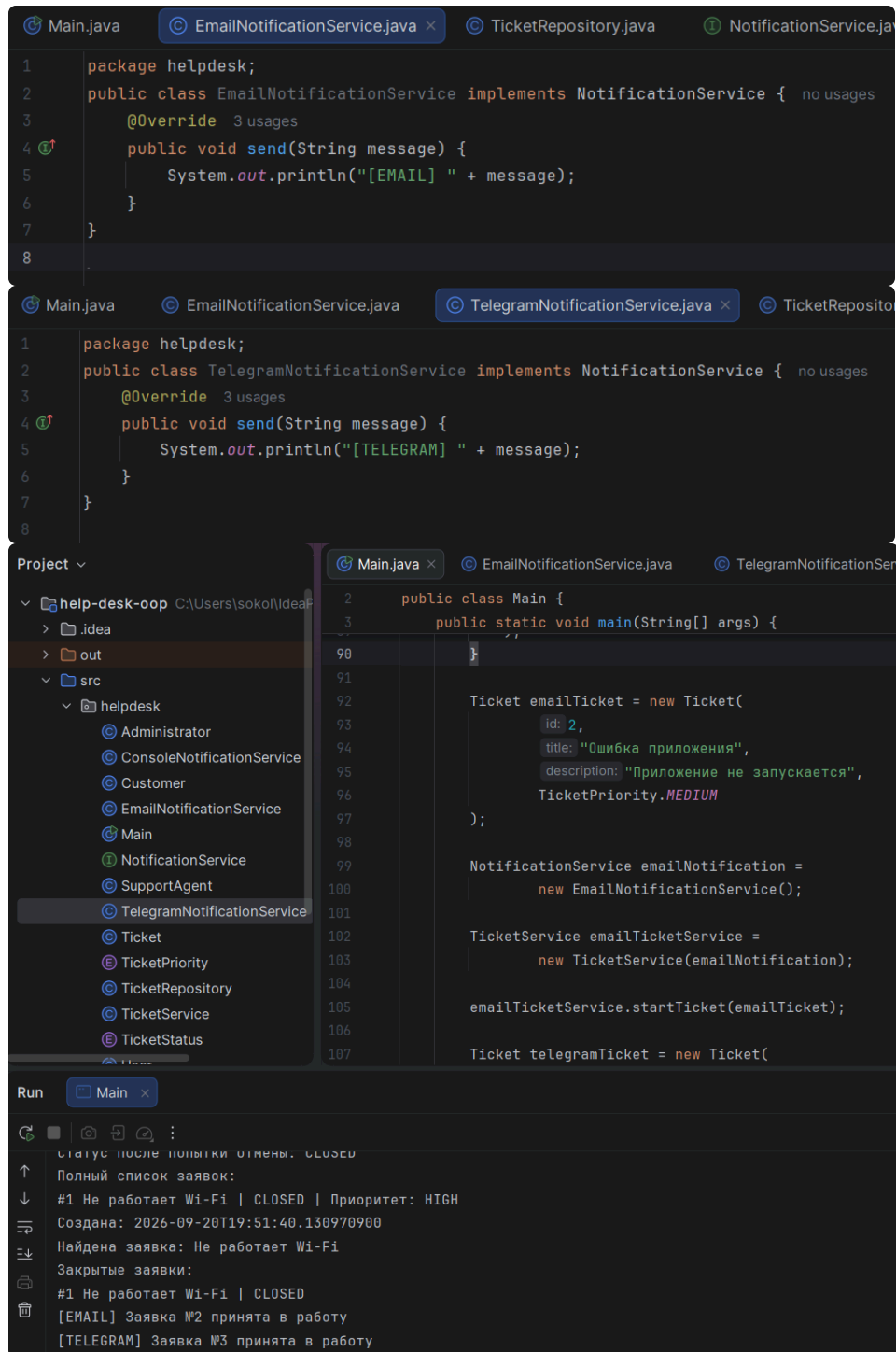
```

=== HELP DESK ===
Клиент: Анна Петрова
Заявка #1: Не работает Wi-Fi | NEW | Приоритет: HIGH
[УВЕДОМЛЕНИЕ] Заявка #1 принята в работу
Статус: IN_PROGRESS
[УВЕДОМЛЕНИЕ] По заявке #1 найдено решение
Статус: RESOLVED
[УВЕДОМЛЕНИЕ] Заявка #1 закрыта
Статус: CLOSED
Ошибка: закрытую заявку нельзя отменить
Статус после попытки отмены: CLOSED
Полный список заявок:
#1 Не работает Wi-Fi | CLOSED | Приоритет: HIGH
Создана: 2026-09-20T19:30:13.964616600
Найдена заявка: Не работает Wi-Fi
Закрытые заявки:
#1 Не работает Wi-Fi | CLOSED
Process finished with exit code 0

```

В `TicketRepository` добавлен метод `findByStatus()`, возвращающий список заявок с указанным статусом.

№10



```
package helpdesk;

public class EmailNotificationService implements NotificationService {
    @Override
    public void send(String message) {
        System.out.println("[EMAIL] " + message);
    }
}

package helpdesk;

public class TelegramNotificationService implements NotificationService {
    @Override
    public void send(String message) {
        System.out.println("[TELEGRAM] " + message);
    }
}

public class Main {
    public static void main(String[] args) {
        // ...

        Ticket emailTicket = new Ticket(
            id: 2,
            title: "Ошибка приложения",
            description: "Приложение не запускается",
            TicketPriority.MEDIUM
        );

        NotificationService emailNotification =
            new EmailNotificationService();

        TicketService emailTicketService =
            new TicketService(emailNotification);

        emailTicketService.startTicket(emailTicket);

        Ticket telegramTicket = new Ticket(
```

Project

- help-desk-oop
- src
- helpdesk
  - Administrator
  - ConsoleNotificationService
  - Customer
  - EmailNotificationService
  - Main
  - NotificationService
  - SupportAgent
  - TelegramNotificationService
  - Ticket
  - TicketPriority
  - TicketRepository
  - TicketService
  - TicketStatus

Run

status после попытки отмены: CLOSED

↑ Полный список заявок:

↓ #1 Не работает Wi-Fi | CLOSED | Приоритет: HIGH

Создана: 2026-09-20T19:51:40.130970900

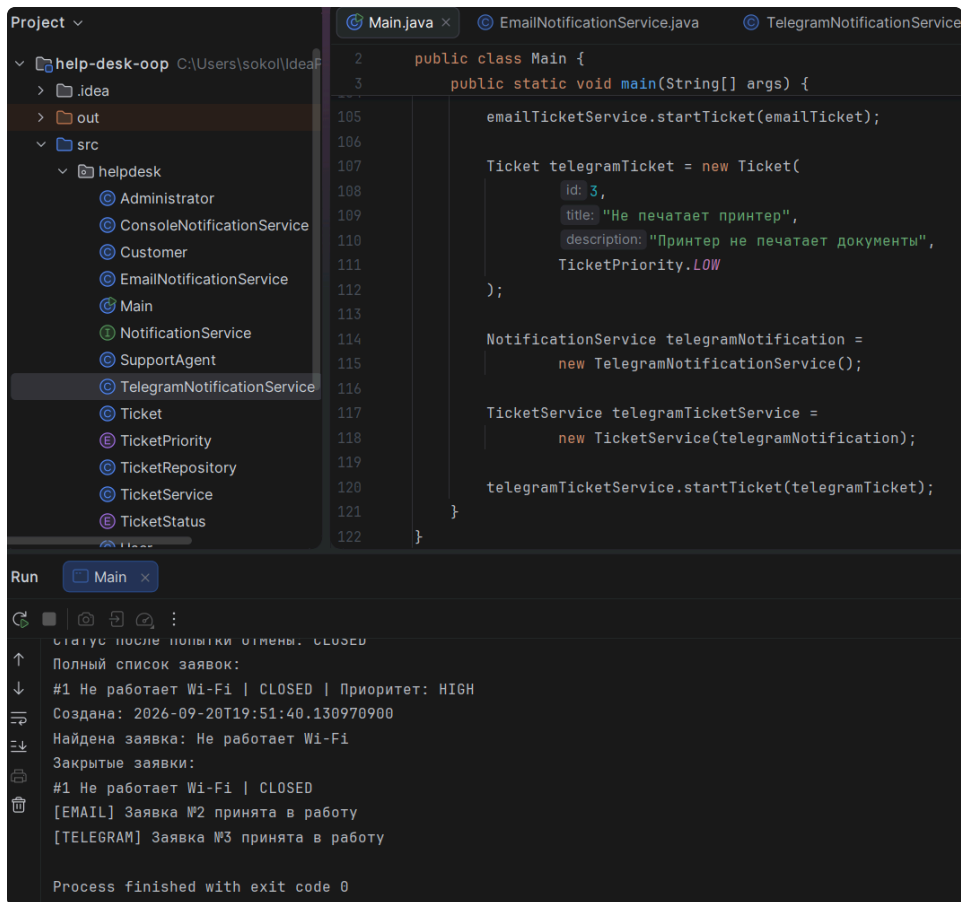
Найдена заявка: Не работает Wi-Fi

Закрытые заявки:

#1 Не работает Wi-Fi | CLOSED

[EMAIL] Заявка №2 принята в работу

[TELEGRAM] Заявка №3 принята в работу



Созданы две дополнительные реализации интерфейса `NotificationService` :

- `EmailNotificationService` ;
- `TelegramNotificationService` .

`TicketService` при этом изменять не потребовалось.

```
Main.java  EmailNotificationService.java  TelegramNotificationService.java  TicketRepository.java

1  package helpdesk;
2  import java.util.ArrayList;
3  import java.util.List;
4  import java.util.Optional;
5  public class TicketRepository { 2 usages
6      private final List<Ticket> tickets = new ArrayList<>(); 5 usages
7      public void add(Ticket ticket) { 1 usage
8          tickets.add(ticket);
9      }
10
11     public List<Ticket> findAll() { 1 usage
12         return new ArrayList<>(tickets);
13     }
14
15     public Optional<Ticket> findById(long id) { 1 usage
16         for (Ticket ticket : tickets) {
17             if (ticket.getId() == id) {
18                 return Optional.of(ticket);
19             }
20         }
21
22         return Optional.empty();
23     }
24
25     public List<Ticket> findByStatus(TicketStatus status) { 1 usage
26         List<Ticket> result = new ArrayList<>();
27
28         for (Ticket ticket : tickets) {
29             if (ticket.getStatus() == status) {
30                 result.add(ticket);
31             }
32         }
33
34         return result;
35     }
36
37     public void getStatistics() { no usages
38         for (TicketStatus status : TicketStatus.values()) {
39             int count = 0;
40
41             for (Ticket ticket : tickets) {
42                 if (ticket.getStatus() == status) {
43                     count++;
44                 }
45             }
46
47             System.out.println(status + ": " + count);
48         }
49     }
50 }
```

The screenshot shows an IDE with the following components:

- Project View:** A tree structure showing the project layout. The 'src' directory contains a 'helpdesk' package with classes: Administrator, ConsoleNotificationService, Customer, EmailNotificationService, and TelegramNotificationService.
- Main.java:** The active file, showing the main method of the Main class. It calls `new TicketService(telegramNotification);`, `telegramTicketService.startTicket(telegramTicket);`, `System.out.println("Статистика:");`, and `repository.getStatistics();`.
- Run Console:** Displays the output of the application. It shows the status of tickets (IN\_PROGRESS, RESOLVED, CLOSED), error messages (e.g., "Ошибка: закрытую заявку нельзя отменить"), and a detailed list of tickets with their status, priority, and creation date. It also shows the statistics of tickets by status.

```
Статус: IN_PROGRESS
[УВЕДОМЛЕНИЕ] По заявке №1 найдено решение
Статус: RESOLVED
[УВЕДОМЛЕНИЕ] Заявка №1 закрыта
Статус: CLOSED
Ошибка: закрытую заявку нельзя отменить
Статус после попытки отмены: CLOSED
Полный список заявок:
#1 Не работает Wi-Fi | CLOSED | Приоритет: HIGH
Создана: 2026-09-20T20:00:28.236614100
Найдена заявка: Не работает Wi-Fi
Закрытые заявки:
#1 Не работает Wi-Fi | CLOSED
[EMAIL] Заявка №2 принята в работу
[TELEGRAM] Заявка №3 принята в работу
Статистика:
NEW: 0
IN_PROGRESS: 0
RESOLVED: 0
CLOSED: 1
CANCELLED: 0
```

В `TicketRepository` добавлен метод `getStatistics()`, который подсчитывает количество заявок по каждому статусу. Полученная статистика позволяет быстро увидеть текущее распределение заявок в системе.

## №19

The screenshot shows an IDE with the following components:

- AuditService.java:** The active file, showing the implementation of the `AuditService` class. It includes a package declaration, imports, and two methods: `log(String action)` and `printAudit()`.

```
package helpdesk;
import java.util.ArrayList;
import java.util.List;
public class AuditService { no usages
    private final List<String> actions = new ArrayList<>(); 2 usages
    public void log(String action) { no usages
        actions.add(action);
    }
    public void printAudit() { no usages
        System.out.println("История действий:");
        for (String action : actions) {
            System.out.println(action);
        }
    }
}
```

```
Main.java x  AuditService.java  EmailNotificationService.java  TelegramNotification

1  package helpdesk;
2  public class Main {
3      public static void main(String[] args) {
4
5          System.out.println("=== HELP DESK ===");
6
7          AuditService auditService = new AuditService();
8
9          Customer customer = new Customer(
10             id: 1,
11             name: "Анна Петрова",
12             email: "anna@mail.ru"
13         );
14
15         SupportAgent supportAgent = new SupportAgent(
16             id: 2,
17             name: "Сергей Иванов",
18             email: "sergey@helpdesk.ru"
19         );
20
21         System.out.println("Клиент: " + customer.getName());
22         auditService.log( action: "Создан клиент: " + customer.getName());
23
24         Ticket ticket = new Ticket(
25             id: 1,
26             title: "Не работает Wi-Fi",
27             description: "После подключения к сети интернет не работает",
28             TicketPriority.HIGH
29         );
30
31         auditService.log( action: "Создана заявка #" + ticket.getId());
32
33         System.out.println(
34             "Заявка #" + ticket.getId()
35             + ": " + ticket.getTitle()
```

```
35         + ": " + ticket.getTitle()
36         + " | " + ticket.getStatus()
37         + " | Приоритет: " + ticket.getPriority()
38     );
39
40     System.out.println("Создана: " + ticket.getCreatedAt());
41
42     NotificationService notificationService =
43         new ConsoleNotificationService();
44
45     TicketService ticketService =
46         new TicketService(notificationService);
47
48     ticketService.startTicket(ticket);
49     System.out.println("Статус: " + ticket.getStatus());
50
51     ticketService.resolveTicket(ticket);
52     System.out.println("Статус: " + ticket.getStatus());
53
54     ticketService.closeTicket(ticket);
55     System.out.println("Статус: " + ticket.getStatus());
56
57     auditService.log( action: "Заявка #" + ticket.getId() + " закрыта");
58
59     ticket.cancel();
60     System.out.println(
61         "Статус после попытки отмены: " + ticket.getStatus()
62     );
63
64     TicketRepository repository = new TicketRepository();
65     repository.add(ticket);
```

```
66
67     System.out.println("Полный список заявок:");
68
69     for (Ticket t : repository.findAll()) {
70         System.out.println(
71             "#" + t.getId()
72             + " " + t.getTitle()
73             + " | " + t.getStatus()
74             + " | Приоритет: " + t.getPriority()
75         );
76
77         System.out.println("Создана: " + t.getCreatedAt());
78     }
79
80     repository.findById(1).ifPresent( Ticket t ->
81         System.out.println(
82             "Найдена заявка: " + t.getTitle()
83         )
84     );
85
86     System.out.println("Закрытые заявки:");
87
88     for (Ticket t : repository.findByStatus(TicketStatus.CLOSED)) {
89         System.out.println(
90             "#" + t.getId()
91             + " " + t.getTitle()
92             + " | " + t.getStatus()
93         );
94     }
95
96     Ticket emailTicket = new Ticket(
97         id: 2,
98         title: "Ошибка приложения",
```

```

99         description: "Приложение не запускается",
100         TicketPriority.MEDIUM
101     );
102
103     NotificationService emailNotification =
104         new EmailNotificationService();
105
106     TicketService emailTicketService =
107         new TicketService(emailNotification);
108
109     emailTicketService.startTicket(emailTicket);
110
111     Ticket telegramTicket = new Ticket(
112         id: 3,
113         title: "Не печатает принтер",
114         description: "Принтер не печатает документы",
115         TicketPriority.LOW
116     );
117
118     NotificationService telegramNotification =
119         new TelegramNotificationService();
120
121     TicketService telegramTicketService =
122         new TicketService(telegramNotification);
123
124     telegramTicketService.startTicket(telegramTicket);
125
126     System.out.println("Статистика:");
127
128     repository.getStatistics();
129
130     auditService.printAudit();

```

Project ▾

- help-desk-oop C:\Users\sokol\IdeaF
  - .idea
  - out
  - src
    - helpdesk
      - Administrator
      - AuditService
      - ConsoleNotificationService
      - Customer
      - EmailNotificationService
      - Main
      - NotificationService
      - SupportAgent

Main.java x AuditService.java EmailNotificationService.java Telegra

```

2     public class Main {
3         public static void main(String[] args) {
4             telegramTicketService.startTicket(telegramTicket);
5             emailTicketService.startTicket(emailTicket);
6
7             System.out.println("Статистика:");
8
9             repository.getStatistics();
10
11             auditService.printAudit();
12         }
13     }
14

```

Run Main x

Создана: 2026-09-20 12:01:16:40.241955900

Найдена заявка: Не работает Wi-Fi

Закрытые заявки:

#1 Не работает Wi-Fi | CLOSED

[EMAIL] Заявка №2 принята в работу

[TELEGRAM] Заявка №3 принята в работу

Статистика:

NEW: 0

IN\_PROGRESS: 0

RESOLVED: 0

CLOSED: 1

CANCELLED: 0

История действий:

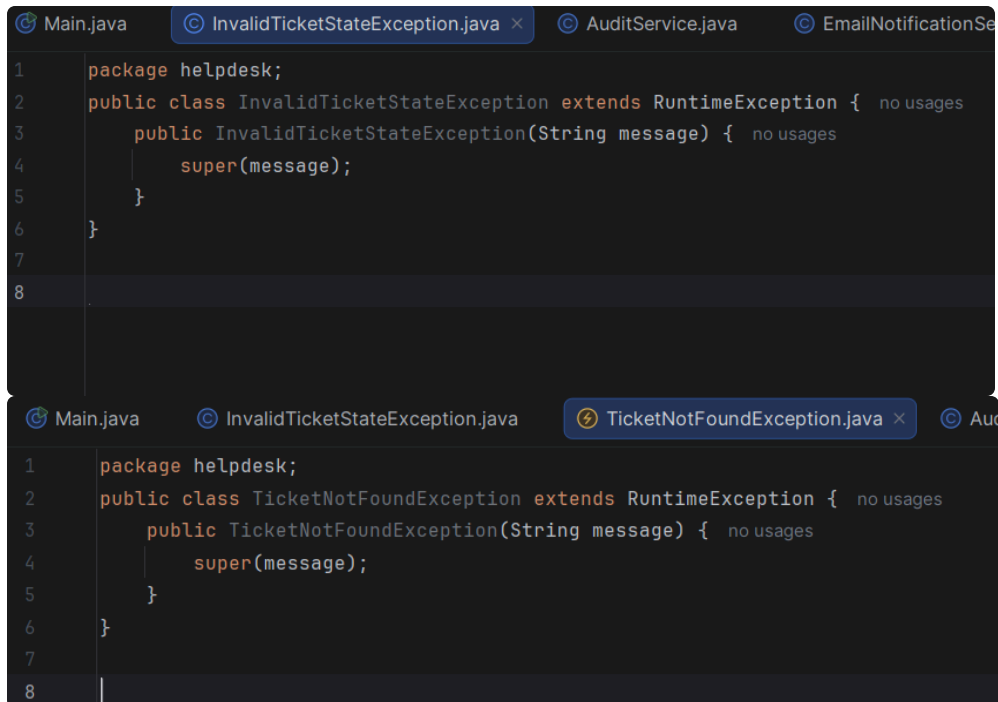
Создан клиент: Анна Петрова

Создана заявка #1

Заявка #1 закрыта

Создан отдельный сервис `AuditService`, который записывает ключевые действия пользователей и системы в список строк.

№16



The screenshot shows an IDE with two open Java files. The top file, `InvalidTicketStateException.java`, contains the following code:

```
1 package helpdesk;
2 public class InvalidTicketStateException extends RuntimeException { no usages
3     public InvalidTicketStateException(String message) { no usages
4         super(message);
5     }
6 }
7
8
```

The bottom file, `TicketNotFoundException.java`, contains the following code:

```
1 package helpdesk;
2 public class TicketNotFoundException extends RuntimeException { no usages
3     public TicketNotFoundException(String message) { no usages
4         super(message);
5     }
6 }
7
8
```

```
tionService.java  TelegramNotificationService.java  TicketRepository.java x  Notific

1  package helpdesk;
2  import java.util.ArrayList;
3  import java.util.List;
4  import java.util.Optional;
5  public class TicketRepository { 2 usages
6      private final List<Ticket> tickets = new ArrayList<>(); 5 usages
7      public void add(Ticket ticket) { 1 usage
8          tickets.add(ticket);
9      }
10
11     public List<Ticket> findAll() { 1 usage
12         return new ArrayList<>(tickets);
13     }
14
15     public Optional<Ticket> findById(long id) { 2 usages
16         for (Ticket ticket : tickets) {
17             if (ticket.getId() == id) {
18                 return Optional.of(ticket);
19             }
20         }
21
22         return Optional.empty();
23     }
24
25     public Ticket getById(long id) { no usages
26         return findById(id).orElseThrow(
27             () -> new TicketNotFoundException(
28                 "Заявка с ID " + id + " не найдена"
29             )
30         );
31     }
}
```

```

32
33     public List<Ticket> findByStatus(TicketStatus status) { 1 us
34         List<Ticket> result = new ArrayList<>();
35
36         for (Ticket ticket : tickets) {
37             if (ticket.getStatus() == status) {
38                 result.add(ticket);
39             }
40         }
41
42         return result;
43     }
44
45     public void getStatistics() { 1 usage
46         for (TicketStatus status : TicketStatus.values()) {
47             int count = 0;
48
49             for (Ticket ticket : tickets) {
50                 if (ticket.getStatus() == status) {
51                     count++;
52                 }
53             }
54
55             System.out.println(status + ": " + count);
56         }
57     }
58 }

```

The screenshot shows an IDE with a project named 'helpdesk'. The project structure includes a 'src' directory with the following classes: Administrator, AuditService, ConsoleNotificationService, Customer, EmailNotificationService, InvalidTicketStateException, Main, NotificationService, and SupportAgent. The 'Main.java' file is open, showing a public class Main with a main method. The main method calls auditService.printAudit(), repository.getById(999), and ticket.resolve(). It catches TicketNotFoundException and InvalidTicketStateException, printing error messages. The Run console shows the output of the application, including email and telegram notifications, statistics, and a list of actions.

```

Project
- helpdesk
  > .idea
  > out
  > src
    > helpdesk
      @ Administrator
      @ AuditService
      @ ConsoleNotificationService
      @ Customer
      @ EmailNotificationService
      @ InvalidTicketStateException
      @ Main
      @ NotificationService
      @ SupportAgent
Main.java
2 public class Main {
3     public static void main(String[] args) {
130         auditService.printAudit();
131         try {
132             repository.getById(999);
133         } catch (TicketNotFoundException e) {
134             System.out.println("Проверка №16: " + e.getMessage());
135         }
136
137         try {
138             ticket.resolve();
139         } catch (InvalidTicketStateException e) {
140             System.out.println("Проверка №16: " + e.getMessage());
141         }
142     }

```

Run Main

```

[EMAIL] Заявка №2 принята в работу
[TELEGRAM] Заявка №3 принята в работу
Статистика:
NEW: 0
IN_PROGRESS: 0
RESOLVED: 0
CLOSED: 1
CANCELLED: 0
История действий:
Создан клиент: Анна Петрова
Создана заявка #1
Заявка #1 закрыта
Проверка №16: Заявка с ID 999 не найдена
Ошибка: решить можно только заявку в работе
Process finished with exit code 0

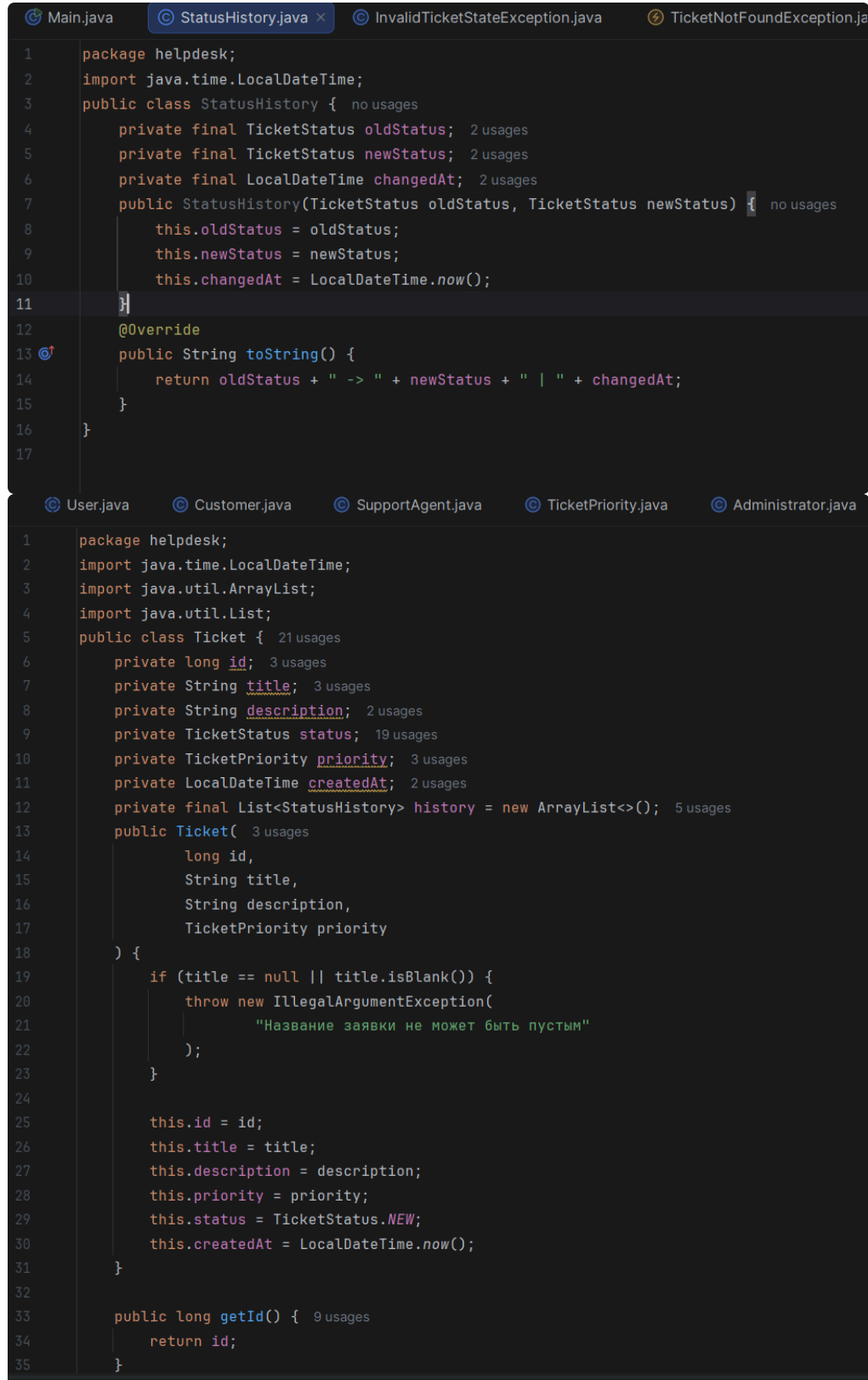
```

Созданы собственные исключения для обработки ошибок:

- `InvalidTicketStateException` ;
- `TicketNotFoundException` .

Они используются вместо части сообщений об ошибках через `println()` .

## №14



```

1 package helpdesk;
2 import java.time.LocalDateTime;
3 public class StatusHistory { no usages
4     private final TicketStatus oldStatus; 2 usages
5     private final TicketStatus newStatus; 2 usages
6     private final LocalDateTime changedAt; 2 usages
7     public StatusHistory(TicketStatus oldStatus, TicketStatus newStatus) { no usages
8         this.oldStatus = oldStatus;
9         this.newStatus = newStatus;
10        this.changedAt = LocalDateTime.now();
11    }
12
13    @Override
14    public String toString() {
15        return oldStatus + " -> " + newStatus + " | " + changedAt;
16    }
17 }

User.java Customer.java SupportAgent.java TicketPriority.java Administrator.java

1 package helpdesk;
2 import java.time.LocalDateTime;
3 import java.util.ArrayList;
4 import java.util.List;
5 public class Ticket { 21 usages
6     private long id; 3 usages
7     private String title; 3 usages
8     private String description; 2 usages
9     private TicketStatus status; 19 usages
10    private TicketPriority priority; 3 usages
11    private LocalDateTime createdAt; 2 usages
12    private final List<StatusHistory> history = new ArrayList<>(); 5 usages
13    public Ticket( 3 usages
14        long id,
15        String title,
16        String description,
17        TicketPriority priority
18    ) {
19        if (title == null || title.isBlank()) {
20            throw new IllegalArgumentException(
21                "Название заявки не может быть пустым"
22            );
23        }
24
25        this.id = id;
26        this.title = title;
27        this.description = description;
28        this.priority = priority;
29        this.status = TicketStatus.NEW;
30        this.createdAt = LocalDateTime.now();
31    }
32
33    public long getId() { 9 usages
34        return id;
35    }

```

```
35     }
36
37     public String getTitle() { 4 usages
38         return title;
39     }
40
41     public String getDescription() { no usages
42         return description;
43     }
44
45     public TicketStatus getStatus() { 9 usages
46         return status;
47     }
48
49     public TicketPriority getPriority() { 2 usages
50         return priority;
51     }
52
53     public LocalDateTime getCreatedAt() { 2 usages
54         return createdAt;
55     }
56
57     public List<StatusHistory> getHistory() { no usages
58         return new ArrayList<>(history);
59     }
60
61     public void startProcessing() { 1 usage
62         if (status != TicketStatus.NEW) {
63             throw new InvalidTicketStateException(
64                 "В работу можно взять только новую заявку"
65             );
66         }
67
68         TicketStatus oldStatus = status;
```

```
69         status = TicketStatus.IN_PROGRESS;
70
71         history.add(new StatusHistory(oldStatus, status));
72     }
73
74     public void resolve() { 1 usage
75         if (status != TicketStatus.IN_PROGRESS) {
76             throw new InvalidTicketStateException(
77                 "Решить можно только заявку в работе"
78             );
79         }
80
81         TicketStatus oldStatus = status;
82         status = TicketStatus.RESOLVED;
83
84         history.add(new StatusHistory(oldStatus, status));
85     }
86
87     public void close() { 1 usage
88         if (status != TicketStatus.RESOLVED) {
89             throw new InvalidTicketStateException(
90                 "Закрыть можно только решённую заявку"
91             );
92         }
93
94         TicketStatus oldStatus = status;
95         status = TicketStatus.CLOSED;
96         history.add(new StatusHistory(oldStatus, status));
97     }
98
99     public void cancel() { 1 usage
100         if (status == TicketStatus.CLOSED) {
101             System.out.println(
```

```
100         if (status == TicketStatus.CLOSED) {
101             System.out.println(
102                 "Ошибка: закрытую заявку нельзя отменить"
103             );
104             return;
105         }
106
107         TicketStatus oldStatus = status;
108         status = TicketStatus.CANCELLED;
109
110         history.add(new StatusHistory(oldStatus, status));
111     }
112
113     @Override
114     public String toString() {
115         return "#" + id
116             + " " + title
117             + " | " + status
118             + " | Приоритет: " + priority;
119     }
120 }
```

Project ▾

- help-desk-oop C:\Users\sokol\Idea
- > .idea
- > out
- src
  - helpdesk
    - Administrator
    - AuditService
    - ConsoleNotificationService
    - Customer
    - EmailNotificationService
    - InvalidTicketStateException
    - Main
    - NotificationService

Main.java x StatusHistory.java InvalidTicketStateException.java Tick

```
2 public class Main {
3     public static void main(String[] args) {
125
126         System.out.println("Статистика:");
127
128         repository.getStatistics();
129
130         auditService.printAudit();
131         System.out.println("История изменений статуса:");
132
133         for (StatusHistory history : ticket.getHistory()) {
134             System.out.println(history);
135         }
136
137     }
```

Run Main x

Статистика:  
NEW: 0  
IN\_PROGRESS: 0  
RESOLVED: 0  
CLOSED: 1  
CANCELLED: 0  
История действий:  
Создан клиент: Анна Петрова  
Создана заявка #1  
Заявка #1 закрыта  
История изменений статуса:  
NEW -> IN\_PROGRESS | 2026-09-20T20:42:51.803803300  
IN\_PROGRESS -> RESOLVED | 2026-09-20T20:42:51.803803300  
RESOLVED -> CLOSED | 2026-09-20T20:42:51.804806100  
Process finished with exit code 0

В проект добавлена история изменения статуса заявки. Для каждого изменения сохраняются старый статус, новый статус и время изменения.

## 6.Творческое самостоятельное задание

## Вариант С. Сервисный центр

```
Main.java  Device.java x  StatusHistory.java  InvalidTicketStateException.java  ⚡

1  package helpdesk;
2  public class Device { no usages
3      private String type; 3 usages
4      private String model; 3 usages
5      private String serialNumber; 3 usages
6      public Device(String type, String model, String serialNumber) { no usages
7          this.type = type;
8          this.model = model;
9          this.serialNumber = serialNumber;
10     }
11     public String getType() { no usages
12         return type;
13     }
14     public String getModel() { no usages
15         return model;
16     }
17     public String getSerialNumber() { no usages
18         return serialNumber;
19     }
20     @Override
21     public String toString() {
22         return type + " | " + model
23             + " | Серийный номер: " + serialNumber;
24     }
25 }
26
```

```
1 package helpdesk;
2 import java.time.LocalDateTime;
3 import java.util.ArrayList;
4 import java.util.List;
5 public class Ticket { 21 usages 3 related problems
6     private long id; 3 usages
7     private String title; 3 usages
8     private String description; 2 usages
9     private TicketStatus status; 19 usages
10    private TicketPriority priority; 3 usages
11    private LocalDateTime createdAt; 2 usages
12    private Device device; 3 usages
13    private final List<StatusHistory> history = new ArrayList<>(); 5 usages
14    public Ticket( 3 usages 3 related problems
15        long id,
16        String title,
17        String description,
18        TicketPriority priority,
19        Device device
20    ) {
21        if (title == null || title.isBlank()) {
22            throw new IllegalArgumentException(
23                "Название заявки не может быть пустым"
24            );
25        }
26
27        this.id = id;
28        this.title = title;
29        this.description = description;
30        this.priority = priority;
31        this.device = device;
32        this.status = TicketStatus.NEW;
33        this.createdAt = LocalDateTime.now();
34    }
35 }
```

```
36     public long getId() { 9 usages
37         return id;
38     }
39
40     public String getTitle() { 4 usages
41         return title;
42     }
43
44     public String getDescription() { no usages
45         return description;
46     }
47
48     public TicketStatus getStatus() { 9 usages
49         return status;
50     }
51
52     public TicketPriority getPriority() { 2 usages
53         return priority;
54     }
55
56     public LocalDateTime getCreatedAt() { 2 usages
57         return createdAt;
58     }
59
60     public Device getDevice() { no usages
61         return device;
62     }
63
64     public List<StatusHistory> getHistory() { 1 usage
65         return new ArrayList<>(history);
66     }
67
68     public void startProcessing() { 1 usage
```

```

68     public void startProcessing() { 1 usage
69         if (status != TicketStatus.NEW) {
70             throw new InvalidTicketStateException(
71                 "В работу можно взять только новую заявку"
72             );
73         }
74
75         TicketStatus oldStatus = status;
76         status = TicketStatus.IN_PROGRESS;
77
78         history.add(new StatusHistory(oldStatus, status));
79     }
80
81     public void resolve() { 1 usage
82         if (status != TicketStatus.IN_PROGRESS) {
83             throw new InvalidTicketStateException(
84                 "Решить можно только заявку в работе"
85             );
86         }
87
88         TicketStatus oldStatus = status;
89         status = TicketStatus.RESOLVED;
90
91         history.add(new StatusHistory(oldStatus, status));
92     }
93
94     public void close() { 1 usage
95         if (status != TicketStatus.RESOLVED) {
96             throw new InvalidTicketStateException(
97                 "Закрыть можно только решённую заявку"
98             );
99         }
100
101         TicketStatus oldStatus = status;

```

```

101         TicketStatus oldStatus = status;
102         status = TicketStatus.CLOSED;
103
104         history.add(new StatusHistory(oldStatus, status));
105     }
106
107     public void cancel() { 1 usage
108         if (status == TicketStatus.CLOSED) {
109             System.out.println(
110                 "Ошибка: закрытую заявку нельзя отменить"
111             );
112             return;
113         }
114
115         TicketStatus oldStatus = status;
116         status = TicketStatus.CANCELLED;
117
118         history.add(new StatusHistory(oldStatus, status));
119     }
120
121     @Override
122     public String toString() {
123         return "#" + id
124             + " " + title
125             + " | " + status
126             + " | Приоритет: " + priority
127             + " | Устройство: " + device;
128     }
129 }

```

```
Main.java x Device.java StatusHistory.java InvalidTicketStateException.java
1 package helpdesk;
2 public class Main {
3     public static void main(String[] args) {
4         System.out.println("=== HELP DESK ===");
5         AuditService auditService = new AuditService();
6         Customer customer = new Customer(
7             id: 1,
8             name: "Анна Петрова",
9             email: "anna@mail.ru"
10        );
11
12        SupportAgent supportAgent = new SupportAgent(
13            id: 2,
14            name: "Сергей Иванов",
15            email: "sergey@helpdesk.ru"
16        );
17
18        System.out.println("Клиент: " + customer.getName());
19        auditService.log(
20            action: "Создан клиент: " + customer.getName()
21        );
22
23        Device device = new Device(
24            type: "Ноутбук",
25            model: "Lenovo IdeaPad 5",
26            serialNumber: "SN123456"
27        );
28
29        Ticket ticket = new Ticket(
30            id: 1,
31            title: "Не работает Wi-Fi",
32            description: "После подключения к сети интернет не работает",
33            TicketPriority.HIGH,
34            device
35        );
```

```
36
37     auditService.log(
38         |         action: "Создана заявка #" + ticket.getId()
39     );
40
41     System.out.println(
42         |         "Заявка #" + ticket.getId()
43         |         + ": " + ticket.getTitle()
44         |         + " | " + ticket.getStatus()
45         |         + " | Приоритет: " + ticket.getPriority()
46     );
47
48     System.out.println(
49         |         "Устройство: " + ticket.getDevice()
50     );
51
52     System.out.println(
53         |         "Создана: " + ticket.getCreatedAt()
54     );
55
56     NotificationService notificationService =
57         |         new ConsoleNotificationService();
58
59     TicketService ticketService =
60         |         new TicketService(notificationService);
61
62     ticketService.startTicket(ticket);
63     System.out.println(
64         |         "Статус: " + ticket.getStatus()
65     );
66
67     ticketService.resolveTicket(ticket);
68     System.out.println(
```

```
69         "Статус: " + ticket.getStatus()
70     );
71
72     ticketService.closeTicket(ticket);
73     System.out.println(
74         "Статус: " + ticket.getStatus()
75     );
76
77     auditService.log(
78         action: "Заявка #" + ticket.getId() + " закрыта"
79     );
80
81     ticket.cancel();
82
83     System.out.println(
84         "Статус после попытки отмены: "
85         + ticket.getStatus()
86     );
87
88     TicketRepository repository =
89         new TicketRepository();
90
91     repository.add(ticket);
92
93     System.out.println("Полный список заявок:");
94
95     for (Ticket t : repository.findAll()) {
96         System.out.println(t);
97     }
98
99     repository.findById(1).ifPresent( Ticket t ->
100         System.out.println(
101             "Найдена заявка: "
```

```
101         "Найдена заявка: "
102         + t.getTitle()
103     )
104 );
105
106 System.out.println("Закрытые заявки:");
107
108 for (Ticket t :
109     repository.findByStatus(TicketStatus.CLOSED)) {
110
111     System.out.println(t);
112 }
113
114 Ticket emailTicket = new Ticket(
115     id: 2,
116     title: "Ошибка приложения",
117     description: "Приложение не запускается",
118     TicketPriority.MEDIUM,
119     new Device(
120         type: "Компьютер",
121         model: "HP ProDesk",
122         serialNumber: "SN222222"
123     )
124 );
125
126 NotificationService emailNotification =
127     new EmailNotificationService();
128
129 TicketService emailTicketService =
130     new TicketService(emailNotification);
131
```

```
132 emailTicketService.startTicket(emailTicket);
133
134 Ticket telegramTicket = new Ticket(
135     id: 3,
136     title: "Не печатает принтер",
137     description: "Принтер не печатает документы",
138     TicketPriority.LOW,
139     new Device(
140         type: "Принтер",
141         model: "HP LaserJet",
142         serialNumber: "SN333333"
143     )
144 );
145
146 NotificationService telegramNotification =
147     new TelegramNotificationService();
148
149 TicketService telegramTicketService =
150     new TicketService(telegramNotification);
151
152 telegramTicketService.startTicket(
153     telegramTicket
154 );
155
156 System.out.println("Статистика:");
157
158 repository.getStatistics();
159
160 auditService.printAudit();
161
162 System.out.println(
163     "История изменений статуса:"
164 );
```

```

166         for (StatusHistory history :
167             ticket.getHistory()) {
168
169             System.out.println(history);
170         }
171
172         try {
173             repository.getById(999);
174         } catch (TicketNotFoundException e) {
175             System.out.println(
176                 "Проверка №16: " + e.getMessage()
177             );
178         }
179
180         try {
181             ticket.resolve();
182         } catch (InvalidTicketStateException e) {
183             System.out.println(
184                 "Проверка №16: " + e.getMessage()
185             );
186         }
187     }
188 }
189 |

```

=== HELP DESK ===

Клиент: Анна Петрова

Заявка #1: Не работает Wi-Fi | NEW | Приоритет: HIGH

Устройство: Ноутбук | Lenovo IdeaPad 5 | Серийный номер: SN123456

Создана: 2026-09-20T21:02:12.945991400

[УВЕДОМЛЕНИЕ] Заявка №1 принята в работу

Статус: IN\_PROGRESS

[УВЕДОМЛЕНИЕ] По заявке №1 найдено решение

Статус: RESOLVED

[УВЕДОМЛЕНИЕ] Заявка №1 закрыта

Статус: CLOSED

Ошибка: закрытую заявку нельзя отменить

Статус после попытки отмены: CLOSED

Полный список заявок:

#1 Не работает Wi-Fi | CLOSED | Приоритет: HIGH | Устройство: Ноутбук | Lenovo IdeaPad 5 | Серийный номер: SN123456

Найдена заявка: Не работает Wi-Fi

Закрытые заявки:

#1 Не работает Wi-Fi | CLOSED | Приоритет: HIGH | Устройство: Ноутбук | Lenovo IdeaPad 5 | Серийный номер: SN123456

[EMAIL] Заявка №2 принята в работу

[TELEGRAM] Заявка №3 принята в работу

Статистика:

NEW: 0

IN\_PROGRESS: 0

RESOLVED: 0

CLOSED: 1

CANCELLED: 0

История действий:

Создан клиент: Анна Петрова

Создана заявка #1

Заявка #1 закрыта

История изменений статуса:

NFW -> IN\_PROGRESS | 2026-09-20T21:02:12.974998200

```
IN_PROGRESS -> RESOLVED | 2026-09-20T21:02:12.975989700
RESOLVED -> CLOSED | 2026-09-20T21:02:12.976994600
Проверка №16: Заявка с ID 999 не найдена
Проверка №16: Решить можно только заявку в работе
```

Теперь к каждой заявке можно привязать устройство клиента.

Устройство содержит:

- тип устройства;
- модель;
- серийный номер.

Связь между классами реализована через композицию: объект `Ticket` содержит объект `Device`.

Для хранения информации об устройстве создан отдельный класс `Device` (хранит характеристики устройства и предоставляет доступ к ним через `getter`-методы.).

Заявка содержит информацию об устройстве, с которым связана проблема.

| Принцип ООП  | Реализация в расширении                                                                                               |
|--------------|-----------------------------------------------------------------------------------------------------------------------|
| Инкапсуляция | Поля <code>Device</code> объявлены как <code>private</code> , доступ осуществляется через <code>getter</code> -методы |
| Композиция   | <code>Ticket</code> содержит объект <code>Device</code>                                                               |
| Абстракция   | Для работы уведомлений используется интерфейс <code>NotificationService</code>                                        |
| Полиморфизм  | <code>TicketService</code> работает с <code>NotificationService</code> , не завися от конкретной реализации           |

## 7.Контрольные вопросы

### 1. Чем класс отличается от объекта?

Класс - это описание свойств и поведения объектов. Объект - конкретный экземпляр этого класса, созданный в программе.

### 2. Почему `private` - поле само по себе ещё не гарантирует хорошую инкапсуляцию?

`private` скрывает поле от прямого доступа извне, но важно ещё правильно организовать методы работы с ним. Например, методы должны не позволять нарушать правила работы объекта.

**3. Почему `setter` для `status` в `Ticket` нежелателен?** Через `setter` можно было бы установить любой статус без проверки. Лучше использовать методы `startProcessing()`, `resolve()` и `close()`, которые контролируют допустимые переходы. **\*\*4. В чём разница между `extends` и `implements`?**

`extends` используется для наследования класса или абстрактного класса. `implements`

используется, когда класс реализует методы интерфейса.

### **5. Что означает отношение IS-A? Приведите пример из Help Desk.**

IS-A означает отношение «является». Например, `Customer` является `User`, поэтому `Customer` наследуется от `User` через `extends`.

### **6. Что означает отношение HAS-A? Приведите пример, который можно добавить в проект.**

HAS-A означает отношение «имеет» или «содержит». Например, `Ticket` может иметь `Device`: заявка содержит информацию об устройстве клиента.

### **7. Зачем используется @Override ?**

`@Override` показывает, что метод переопределяет метод родительского класса или реализует метод интерфейса. Компилятор проверяет, что переопределение выполнено правильно.

### **8. Почему List<User> может хранить Customer и SupportAgent ?**

Потому что `Customer` и `SupportAgent` являются наследниками `User`.

### **9. Как Java выбирает конкретную реализацию performAction() во время выполнения?**

Java определяет фактический тип объекта и вызывает переопределённый метод этого класса.

### **10. Почему абстрактный класс User нельзя создать через new ?**

Абстрактный класс предназначен для использования как базовый класс и может содержать абстрактные методы без реализации.

### **11. Чем интерфейс NotificationService полезнее жёсткой зависимости от ConsoleNotificationService ?**

`TicketService` зависит от общего интерфейса, а не от конкретного класса. Поэтому реализацию можно заменить, например, на `EmailNotificationService` или `TelegramNotificationService` без изменения `TicketService`.

### **12. Почему enum TicketStatus надёжнее строкового статуса?**

`enum` задаёт ограниченный набор допустимых значений: `NEW`, `IN_PROGRESS`, `RESOLVED`, `CLOSED` и т.д. Это защищает от опечаток и случайных неправильных значений.

### **13. Какой класс должен отвечать за допустимые переходы статуса: Main, TicketService или Ticket ? Обоснуйте.**

За переходы должен отвечать `Ticket`, потому что именно он хранит своё состояние и контролирует его изменение. Это соответствует инкапсуляции: объект сам не допускает некорректных переходов.

### **14. Что изменится в архитектуре при переходе с консольных уведомлений на Telegram?**

Нужно создать новую реализацию интерфейса `NotificationService`, например `TelegramNotificationService`. `TicketService` при этом менять не нужно, потому что он работает с интерфейсом.

### **15. Где в созданном проекте можно использовать композицию вместо**

### **наследования?**

В проекте это можно использовать между Ticket и Device : заявка имеет устройство. Device не является разновидностью Ticket , поэтому здесь используется отношение HAS-A, а не наследование.